The GIL, Not the Syscall: What Batching Buys a Python `io_uring` Runtime on Multiple Cores

Marimuthu Velayutham*

October 7, 2026

Abstract

Linux `io_uring` is being adopted in Python event loops in the hope that fewer system calls make them faster. This measurement study asks what batching buys when worker threads share one CPython interpreter and its Global Interpreter Lock (GIL). On a four-core server under saturating load, the answer is mostly fewer GIL hand-offs. An `epoll` reactor in C that calls a one-microsecond Python handler delivers, with four threads, 0.70× the throughput of one when it takes the GIL per request and 2.33× when it takes it once per batch, with the I/O system calls and the handler unchanged. One process per worker reaches 2.41×. On `io_uring` the same change raises scaling from 1.61× to 2.18×, in steps as more requests are served per acquisition. A Python loop on `epoll` that releases the GIL at every system call reaches 0.62×; issuing the calls of a pass through one C call that releases it once gives 2.35×. Saved system calls matter much less: on one worker the interface and its batching are worth 1.15–1.25×. We describe `uringpy`, a runtime whose workers reap completions in C and take the GIL once per batch. Its benefit is narrow. Threads reach 0.93 of the throughput of its processes with the one-microsecond handler and 0.63 with a four-microsecond one, batching needs a backlog, and on the free-threaded build threads scale without it when handlers are short. Hand-offs were inferred from counters and interventions, not observed directly. Code, raw data and scripts are public.

Index Terms— io_uring, asynchronous I/O, Python, Global Interpreter Lock, event loop, multicore scaling, system-call amortization

1 Introduction

Linux `io_uring` [1] lets a program queue many I/O operations and collect many results with a single system call. With `epoll`, by contrast, the kernel only reports which sockets are ready, and the program still makes one system call for each read and each write. Python projects have started to put `io_uring` underneath an `asyncio`-compatible event loop [2, 3]. Where such a loop is reported to be faster than the standard one, the gain is credited to taking system calls out of the hot path [3].

That explanation is plausible, but it is hard to check from an end-to-end benchmark, because three things change at once when one event loop is swapped for another. The number of system calls changes. So does the amount of work done per event, since some loops dispatch each event in C and others in Python. And so does the way the loop treats CPython's Global Interpreter Lock (GIL), which allows only one thread to run Python code at a time. Every time a worker thread gives the lock up it has to get it back, and when other threads want the lock too, getting it back can mean being put to sleep and woken again. A typical `epoll` loop written in Python makes more system calls, runs more bytecode and releases the GIL more often than a C loop on `io_uring`, so comparing the two cannot tell us which of these differences is responsible.

In this paper we separate them. We wrote the same minimal server loop four times, on `io_uring` and on `epoll`, once in C with the GIL released and once in Python. We then added two controls, each aimed at one of the confounded factors. The first is the Python `epoll` loop with the `recv` and `send` calls of a pass issued by a single C call. This leaves the system calls as they were and releases the GIL once for all of them; it also takes most of the per-event work out of the interpreter. The second is the Python `io_uring` loop modified to hold on to the GIL while it submits. The cleanest comparisons come from two reactors written in C that call a Python handler and take the GIL either for every request or once per batch of requests.

With a single worker, both the interface and the language of the loop make a modest difference. In the 2×2 design `io_uring` is 1.15–1.25 times faster than `epoll`, and a C loop is 1.08–1.17 times faster than the equivalent Python loop. With several worker threads in one interpreter the picture changes, and what matters is how often the GIL changes hands. The Python loop on `epoll` gives the lock up 2.012 times per request, and four threads of it manage only 0.62 times the throughput of one. Keeping the interface and the 2.013 system calls per request, but cutting the releases to 0.026 per request, lifts that to 2.35 times. The Python loop on `io_uring` reaches 2.19 and the two C loops 2.28 and 2.36, which is as much as the C loops reach as four processes on our four-core server. A

*M. Velayutham is an independent researcher (e-mail: marimuthu.vm@gmail.com; ORCID: 0009-0007-7352-5552).

C reactor that calls a Python handler gives the cleanest version of the result, because there the Python executed per request stays the same. On `epoll`, four threads deliver 0.70 times the throughput of one when the GIL is taken for each request and 2.33 times when it is taken once per batch. So batching does help worker threads that share an interpreter, but mostly because it reduces GIL hand-offs. The saved system calls are a smaller effect, and a completion-based interface is one convenient way of getting the batching, not a requirement for it.

We used this observation to build `uringpy`, a small runtime that reaps completions in C. It keeps hand-offs rare in three ways. Its basic reactor runs the whole accept, receive and send cycle inside a single `nogil` region and never executes Python. A second mode calls a Python handler for each request but takes the GIL once for a whole batch of requests. Finally, it exposes a batch interface on which a loop can be written in Python. Each worker has its own ring and its own `SO_REUSEPORT` listener. With no Python per request, four worker threads of one interpreter then deliver what four processes do on an ordinary GIL build. The window in which threads keep up once Python runs is narrow, and we measured its edges. As the handler's hold on the lock grows from one microsecond to four, the threads go from 0.93 to 0.63 of what its processes deliver. Under light load there is little to batch. On the free-threaded build `asyncio` threads scale without help.

The study has clear limits, and we state them here. Everything was measured on one four-core virtual machine under saturating closed-loop load, with synthetic handlers. GIL hand-offs are inferred from counters and from interventions; we did not trace the lock. And no existing `io_uring` event loop is evaluated: the two we tried could not be benchmarked on CPython 3.14 (Section 7).

The paper makes three contributions.

- A framing of thread scaling under the GIL in two quantities, the time a request holds the lock and the number of times it hands the lock over, formulated after our first measurements, together with experiments that vary the system-call interface, the placement of the loop and the number of GIL releases (Sections 3 and 5).
- `uringpy`, a runtime that reaps completions and runs the connection protocol in C, and that takes the GIL once per batch when it calls a Python handler. With a one-microsecond handler this raises thread scaling at four workers from 1.61× to 2.18× (Sections 4 and 5).
- An evaluation that covers threads and processes on both the GIL and the free-threaded build, with and without a Python handler, across connection counts and response sizes. We report confidence intervals and processor usage for the main results, including those that contradict the hypothesis we started from (Section 5).

The rest of the paper is organised as follows. Section 2 reviews related work. Section 3 develops the cost model, and Section 4 describes the runtime. Section 5 presents the experiments and their results. Section 6 discusses what they imply, Section 7 lists the threats to validity, and Section 8 concludes.

2 Background and Related Work

Reducing system calls. The cost of entering the kernel for every I/O operation has been studied for a long time. FlexSC [4] showed that issuing system calls in batches and running them asynchronously removes much of the overhead of switching modes, and MegaPipe [5] combined batching with per-core listening sockets. `io_uring` [1] brought the idea into mainline Linux through a pair of rings shared between the application and the kernel. Most measurements of `io_uring` so far concern storage. Didona et al. [6] compared it with `libaio` and SPDK and found that its performance depends heavily on how it is configured. Jasny et al. [7] found that dropping it into an existing database system brings little, while designing around its asynchronous, batched model brings a lot. Both studies caution against crediting a speed-up to the interface alone. We ask the same question for a network server in a managed language, where an interpreter lock is involved in handling every event.

The GIL. CPython runs bytecode under a global lock. Beazley [8] described how the lock is passed between threads and how badly I/O-bound threads can fare when others compete for it, which is the behaviour at the centre of this paper. It is an instance of the lock convoy that Blasgen et al. [9] described for database systems. Three ways around the lock exist. Sub-interpreters can each have their own GIL [10]; like worker processes they do not share objects, and we use processes as our reference for that design. The second is to remove the lock: the free-threaded build [11] is officially supported, though not the default, from Python 3.14 [12], and the standard event loop has been reworked to scale on it [13]. This is why we evaluate on CPython 3.14 both with and without the GIL. The third is to move work into native code that releases the lock, which Cython [14] makes straightforward with `nogil` blocks, and which is what we do. Whatever still runs under the lock limits scaling in the way Amdahl described [15], and Eyerman and Eeckhout [16] extended that argument to contended critical sections. Section 3 adapts both to the GIL.

That contention for the GIL hurts I/O-bound threads has therefore been known for some time. What has been missing is a measurement that ties it to the design of an event loop. This paper supplies upper limits on the hand-offs that different loop designs allow per request, and the scaling that comes back when the releases are batched, and uses them to separate the effect of the lock from that of the system-call interface. It does not measure the cost of a single hand-off.

Python event loops and servers. `uvloop` [17] replaces the `asyncio` loop with one built on libuv and is the usual fast baseline. `uringloop` [2] and `uringcore` [3] are `asyncio` loops backed by `io_uring`. `uringcore` reports a throughput 36% above `asyncio`'s in its own benchmark and attributes it to its completion-driven design, which removes system calls from the hot path; `uringloop` makes no performance claim. Both deliver events to Python callbacks, as any `asyncio` loop must. Doing network I/O outside the interpreter is not new either. Granian [18], for instance, runs it on Rust threads and enters Python only for the application. Our runtime belongs to the same family, but it is deliberately small, so that we could pair it with an otherwise identical `epoll` reactor and with Python loops on both interfaces. That pairing is what lets us separate the interface, the placement of the loop and the GIL hand-offs. We tried to benchmark the two `io_uring` loops cited above and could not (Section 7), so we make no claim about how either behaves. Our workers follow the usual shared-nothing layout, with one event loop and one `SO_REUSEPORT` listener per worker, as MegaPipe proposed for per-core accept queues [5].

Measurement. Benchmarks are easy to get wrong when runs are few or the setup is uncontrolled [19]. We repeat every configuration, interleave the repetitions, and report means with confidence intervals alongside processor usage on both machines. Load comes from `wrk` [20], which is a closed-loop generator: it waits for each response before sending the next request on a connection. Schroeder et al. [21] showed how much results can differ between closed and open systems. In particular, a closed-loop client slows down with the server, so its latency figures understate what an open-loop client would see, and our throughput figures describe servers under saturating load. We use the latency figures only to compare configurations under the same load.

3 A Cost Model for Worker Threads under the GIL

Table 1 lists the symbols used in the rest of the paper. The model of this section is a framing and not a predictive theory. We formulated it after our first measurements, to explain them (Section 5.4), and the experiments that test its direction came afterwards. It has one term we can measure, the time under the lock, and one we can only bound from above and infer, the hand-offs.

A bound from time under the lock. Suppose a request needs t_c of processor time that does not touch interpreter state, and can therefore run with the GIL released, plus t_p that has to run with the GIL held. One saturated worker then serves $X_1 = 1/(t_c + t_p)$ requests per second. Now run N worker threads in one interpreter on at least N cores. Each worker still spends $t_c + t_p$ on every request, so $X_N \le N/(t_c+t_p)$. In addition, only one thread can hold the GIL at any moment, so all requests together can occupy it for at most one second per second: $X_N\, t_p \le 1$. Dividing by X_1 gives

$$S_N = \frac{X_N}{X_1} \le \min\left(N,\ \frac{1}{f}\right), \qquad f = \frac{t_p}{t_c + t_p}, \quad (1)$$

where f is the fraction of a request's processor time spent holding the GIL. This is Amdahl's law [15] with the GIL-held portion playing the serial part. Two extreme cases are worth keeping in mind. A loop that holds the GIL for everything it does has $f = 1$ and can never beat a single worker, however many threads it runs. A loop that never enters the interpreter has $f = 0$ and is limited only by N. A loop written in Python can sit anywhere in between, because the system calls it makes with the GIL released count towards t_c. When each worker is its own process, each has its own GIL, the second constraint disappears, and $S_N \le N$ for any f. We therefore treat process-mode results as the reference for what the hardware can deliver.

Table 1: Notation.

N	number of workers (threads of one interpreter, or processes)
X_N, $\bar{x}_N$	throughput of N workers in requests per second: in the model, and as the mean of the measured runs
S_N	scaling, X_N/X_1 in the model and $\bar{x}_N/\bar{x}_1$ as measured
t_p	time per request spent holding the GIL ($t_{p,N}$: measured with N workers)
t_c	processor time per request spent with the GIL released
f	share of a request spent holding the GIL, $t_p/(t_c + t_p)$
g	GIL hand-offs per request: times the lock passes to another thread on behalf of one request. Not observed directly
$\hat{g}$	counted GIL releases (loops in Python) or acquisitions (C reactors) per request; an upper limit on g
τ	cost of one hand-off when another thread is waiting
w	time a request waits to acquire the GIL
U_N	GIL utilisation, $\bar{x}_N\, t_{p,N}$: share of each second the lock is held running request code
C_N	busy server cores
E_N	throughput per busy core relative to one worker

The cost of handing the lock over. Equation (1) only accounts for the time the GIL is held. Passing it from one thread to another is not free: the thread that releases it has to wake a waiter, and that waiter has to be scheduled before it can do anything. Eyerman and Eeckhout model costs of this kind for critical sections in general [16]. Say a request causes g hand-offs, and suppose the lock does no useful work for a time τ at each of them. The GIL is then tied up for $X_N\,(t_p + g\,\tau)$ of every second, which gives

$$X_N \le \frac{1}{t_p + g\,\tau}, \qquad S_N \le \min\left(N,\ \frac{1}{f + g\,\tau\,X_1}\right). \quad (2)$$

The number g depends on how the server is built and not on the request. Every release of the GIL is an opportunity for a hand-off, so the number of releases per request, which we write $\hat{g}$, is an upper limit on g. A loop that releases the GIL around every system call can hand it over once per system call. A C reactor that takes the GIL

for each handler call can hand it over once per request, and a loop that gives the GIL up once per batch of B requests only once in B requests. Whether an opportunity becomes a hand-off depends on whether another thread is waiting, and on whether that thread gets to run before the one that released the lock asks for it again. This is the only place where the system-call interface comes into the model, and it does so indirectly. A completion interface hands back many completions per call, so a loop built on it naturally releases the GIL once per batch. A readiness loop written in the usual way releases it at every read and every write. Nothing forces that, though. A readiness loop can batch its releases too, and one of the loops in Section 5 does exactly this.

Equation (2) is a bound that predicts a direction; it is not a calibrated formula. The cost τ is not a constant. It depends on whether anyone is waiting, on how quickly the kernel schedules the waiter and on the interpreter's switch interval, and in Section 5 we find that no single value fits all our configurations. What the model does predict, and what the experiments test, is an ordering. If f stays the same, scaling should improve as the releases become rarer, whatever the interface and whatever language the loop is written in. And once they are rare, scaling should be limited by $1/f$.

What we measure, and what we do not. We measure t_p and the time w that a request waits for the GIL, by timing every handler call. Both are elapsed times, so t_p includes any time for which the thread holding the lock was interrupted. We count $\hat{g}$: GIL releases in the Python loops and GIL acquisitions in the C reactors. The count does not include releases that the interpreter forces on a thread that has held the lock for a full switch interval (5 ms), which matter only for our longest handlers. We do not measure g or τ: the lock itself is not traced. For the Python loops, whose time under the lock we did not instrument, f is estimated from throughput. As an indirect sign of hand-offs we record the voluntary context switches of the server process. A thread makes one whenever it goes to sleep, whether for the lock or for I/O, so under a load that leaves no worker idle they indicate how many of the opportunities ended with a thread asleep. From t_p we compute the *GIL utilisation* $U_N = \bar{x}_N\, t_{p,N} \leq 1$, the share of each second in which the GIL is held running request code. In the remainder, $1 - U_N$, the lock is either changing hands or idle.

A probe without I/O. The two extremes of (1) can be seen without a server. We run the same arithmetic over synthetic events in an interpreted loop ($f = 1$) and inside a single C `nogil` region ($f = 0$), while one CPU-bound Python thread competes for the GIL. The interpreted loop then takes 2.06 times as long as it does alone and the C loop 1.02 times as long (20 repetitions). With two or three competing threads the timings of both loops scatter widely on our four-core virtual machine, and the script's own criterion, which compares means with three competitors, is not met; we do not use those rows (the full output is in the repository). The probe involves no I/O and says nothing about the interface or about hand-offs.

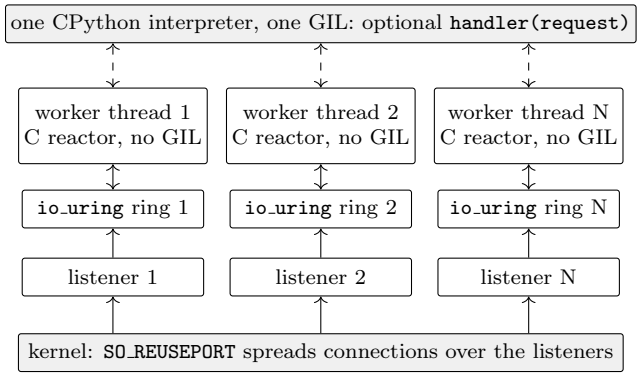


Figure 1: A server is N engines in one process. Solid arrows are the data path, which never leaves C; the dashed arrows mark the only point where a worker takes the GIL, and exist only in application mode.

4 Design

`uringpy` is a Cython [14] extension built on `liburing`. Its basic unit is the *engine*, which consists of one `io_uring` instance, a pool of fixed-size receive buffers and a reactor loop. A server is made of N engines. Each runs on its own OS thread and has its own `SO_REUSEPORT` listening socket on the shared port, so the kernel distributes incoming connections among the workers (Fig. 1). The engines share nothing in user space. No ring, buffer or connection is ever touched by two workers, and so the data path needs no user-level locks. Rings are created with default flags, and we use plain single-shot accept, receive and send operations without registered buffers or files. The interface effects we report in Section 5 therefore apply to this basic way of using `io_uring`.

The loop. Fig. 2 shows the loop that each worker runs, all of it inside one `nogil` region. Every iteration makes exactly one system call, `io_uring_submit_and_wait`. That call passes to the kernel whatever operations were queued while the previous batch was being handled, and returns once at least one completion is available. The loop then works through all the completions already sitting in the ring without going back to the kernel. Any follow-up operations, such as the next receive or the send of a response, are written to the submission ring and go out with the next iteration's call. No Python object is created for a completion, and the interpreter is never entered.

Per-operation state. Each operation carries a 64-bit tag, which the kernel returns with the completion. Eight bits give the operation type, 32 the file descriptor, and the remaining 24 hold either the receive-buffer slot (for a receive) or the number of response bytes already sent (for a send). Dispatch needs no lookup table as a result. Because the offset travels with the operation, a send that

```
1: queue ACCEPT on the listener
2: while not stopped do
3:   io_uring_submit_and_wait(1) {one system call}
4:   for each completion q in the ring, up to the batch
     cap do
5:     if q is an ACCEPT then
6:       queue ACCEPT; queue RECV on the new con-
         nection
7:     else if q is a RECV of n > 0 bytes then
8:       if application mode then
9:         take GIL; r ← handler(request)
10:        copy r to the connection's buffer; release
           GIL
11:      end if
12:      queue SEND of the response from offset 0
13:    else if q is a SEND of n > 0 bytes then
14:      if part of the response is still unsent then
15:        queue SEND of the remainder
16:      else
17:        count one request; queue RECV
18:      end if
19:    else
20:      close the connection {peer closed, or error}
21:    end if
22:  end for
23: end while
```

Figure 2: The reactor loop of one worker. Queued operations reach the kernel at the next execution of line 3. With GIL batching, lines 9–10 are deferred: the requests of a pass are collected and the GIL is taken once for all of them.

the kernel completes only partially can be resumed where it stopped, and a response may be larger than a buffer slot (up to 16 MiB). Receive buffers come from a per-engine slab allocator. If no buffer or submission entry is available for a connection, the connection is closed instead of being left hanging.

Two entry points. `serve_forever_echo` replies to every request with a fixed response and never takes the GIL, so it measures the transport path by itself. `serve_forever_app` hands each request to a Python callable. The reactor takes the GIL only for that call. It builds the request `bytes`, invokes the handler, copies the result into a per-connection buffer that the engine owns, and releases the GIL before queueing the send. Accept, receive and send stay outside the GIL in both modes.

By default the GIL is taken once per request, which gives another worker the chance to take it once per request ($\hat{g} = 1$). With *GIL batching* switched on, the reactor instead collects all the requests it finds in one pass over the completion queue and takes the GIL once for the lot. The handler sees the same calls in the same order. The difference is that the lock changes hands once per batch, and the cost is that no response from a batch is queued until every handler in it has finished. An optional cap limits how many requests are handled under one acquisition. This bounds the delay, and it also lets us vary $\hat{g}$ between 1 and the reciprocal of the batch size in the experiments. If a handler raises an exception, that connection is closed and the error is counted. An interrupt such as `KeyboardInterrupt` or `SystemExit` stops the reactor and is re-raised in the thread that started it. Neither entry point parses HTTP. Each replies once per `recv`, so pipelined and fragmented requests are out of scope, and the same is true of every baseline we compare against.

A batch interface for Python. The engine also makes submission and reaping available to Python code. Operations are queued without a system call, `flush` submits everything that is queued, and `get_events` returns the completions that have arrived as a list. Both calls release the GIL around their system call, so a Python loop written on this interface gives the GIL up twice per batch no matter how many requests the batch holds. One detail turns out to matter a great deal. The kernel may carry out queued operations, a send for example, inside the submitting call itself. Whether the GIL is held across that call therefore determines how much of a request's processor time is spent under the lock.

Instrumentation. The reactor counts its `io_uring_enter` calls, its completions and the responses it has fully sent, which means system calls per request and completions per system call are measured and not estimated. A batch cap limits the completions processed per system call so that batching can be varied by itself. GIL acquisitions for handler calls are counted, and optional timing records how long each handler call waited for the GIL and how long it held it. The clock is `CLOCK_MONOTONIC`. The wait runs from just before the reactor asks for the GIL until it has it, and the hold from that moment until the handler has returned and its result has been copied, just before the release. With GIL batching there is one wait and one hold per acquisition, and both are divided by the number of handler calls, so the figures stay comparable per request. The counters are plain integer increments, and the timing is off unless requested.

An `epoll` twin. For the comparisons in Section 5 the package also includes `EpollEngine`, which is the same loop written for `epoll`. It makes one `recv` and one `send` system call per request, replies once per `recv`, resumes partial sends, runs inside one `nogil` region and counts every system call. It offers the same two entry points and the same GIL batching: in application mode it reads from every ready socket, takes the GIL once to run the handlers for that pass, and then sends the responses. There is also a small helper, `BatchIO`, for Python loops on `epoll`. The loop passes it the sockets that `epoll` reported as ready, and a single call then performs the `recv` and `send` for each of them with the GIL released the whole time. These are the same system calls the loop would have made through CPython's socket objects. All that changes is how often

the GIL is released, which drops from once per call to once per pass.

Interpreter integration. When a signal interrupts a wait, control returns to the interpreter for long enough to run any pending Python signal handlers, so a worker on the main thread can be stopped in the normal way. The extension declares itself safe for free-threaded builds, which means loading it there does not switch the GIL back on. The benchmark image checks this when it is built.

5 Evaluation

5.1 Setup

Table 2 describes the testbed. The server and the load generator are separate virtual machines in the same zone. Both belong to a machine family in which every vCPU is a full physical core[1], so four workers have four physical cores between them. The client has twice as many cores as the server so that it does not become the bottleneck. The code under test runs in a container on the server with host networking. Both interpreters are built from one recipe that differs only in the `--disable-gil` flag. Comparing them is therefore not also a comparison of compilers or build options, and every run records the interpreter version and the GIL state reported by the server process.

All results in this paper were measured with the server code of a single commit (`5db4eab`). The main run was made by a single script. It sends that commit to the server, builds both images from it, runs the test suite inside each image (61 tests, which passed) and then runs the experiments one after another. Conditions that could differ between engines are set in one place. In particular, every listening socket has a backlog of 1024 and `TCP_NODELAY` set, and accepted connections inherit the latter. Workers are not pinned to cores, and interrupt and receive-queue settings are those of the stock image.

Two measurements were added on the same day by a second script, because two limits of the server's container had turned out to be too low. A process in it could hold 1024 open files, fewer than the 1600 connections of one cell of the main run, and lock 8 MiB of memory, too little for `uringcore`, as a first attempt to start it showed. This supplementary run uses the same images, so the server code is unchanged. The script (driver commit `abf3e7c`) raises the two limits and records them with every result. It supplies the 1600-connection rows of Table 11 and the handler sizes of 5, 10 and 20 iterations in Table 9 and Fig. 5. A third part of it, an attempt to measure `uringcore` in an image derived from the same one, failed (Section 7); the limit on locked memory matters only for that loop. Its repetitions of `asyncio` and `uvloop` are the only configurations measured in both runs: over those 12 configurations the supplementary run gave between 0.97 and 1.01 times the throughput of the main run.

[1] Google Cloud, "CPU platforms," Compute Engine documentation, `https://docs.cloud.google.com/compute/docs/cpu-platforms`.

Table 2: Testbed.

Server	Google Compute Engine `t2d-standard-4`: 4 vCPUs (AMD EPYC 7B13), one per physical core, 16 GB
Client	`t2d-standard-8`: 8 vCPUs (AMD EPYC 7B13), one per physical core; same zone (`us-east1-b`)
System	Ubuntu 24.04 LTS, Linux 7.0.0-1011-gcp on both
Runtime	Docker, Debian 12 base, `liburing` 2.6; CPython 3.14.7 built from source with and without `--disable-gil`; `uvloop` 0.23.0
Load	`wrk` 4.1.0, 8 threads, 400 keep-alive connections unless stated, `GET /`
Response	13-byte body on the transport path; 19-byte JSON body built by the handler; other sizes where stated
Run	fresh server process; 5 s warm-up (discarded), 20 s measured
Repetitions	5 per configuration, interleaved in shuffled order

5.2 Engines

Table 3 lists the servers we compare. They all answer each `recv` with the same bytes and none of them parses HTTP, so the only differences are in how the I/O loop is built and when the GIL is held. Every engine runs as N workers with one `SO_REUSEPORT` listener each, either as threads of one interpreter (*thread* mode, one GIL) or as separate processes (*process* mode, one GIL per worker).

The first four engines make up a 2×2 design that varies the system-call interface and the placement of the per-event loop. The two Python loops in it are bare dispatch loops written directly against the interface. They have none of `asyncio`'s tasks, futures and transports, and nobody would deploy them as they are. The design has a confound, and it turns out to be the thing that matters most. `py-epoll` makes its `recv` and `send` calls through CPython's socket objects, which release the GIL around every system call. `py-uring` uses the batch interface of Section 4 and releases the GIL around its two calls per pass. Going from one cell to the other therefore changes the interface, the number of system calls and the number of GIL releases all at once.

We added two controls to pull these apart. Each is aimed at one of the confounded quantities, and neither is a cell of the design. `py-epoll-batch` is `py-epoll` with the `recv` and `send` calls of each pass issued through `BatchIO`, which keeps the interface and the system calls but releases the GIL once per pass for all of them. Its loop over the ready sockets is still Python, but the per-socket work is not. `py-uring-held` is `py-uring` changed to keep the GIL while it submits. Since the kernel performs the queued sends inside that call, the GIL is released no more often than before (half as often, in fact), but most of a re-

Table 3: Engines. "C" means the loop runs with the GIL released; the handler is the same Python function for every engine that calls one.

Engine	Interface	Per-event loop and GIL
`uringpy`	`io_uring`	C
`c-epoll`	`epoll`	C (same loop)
`py-uring`	`io_uring`	Python; GIL released per pass
`py-epoll`	`epoll`	Python; GIL released per syscall
`py-epoll-batch`	`epoll`	as `py-epoll`, GIL released per pass
`py-uring-held`	`io_uring`	as `py-uring`, GIL held in submit
`asyncio`	`epoll`	Python, streams API
`asyncio-proto`	`epoll`	Python, Protocol API
`uvloop(-proto)`	libuv	C loop, Python callbacks
`uringpy-app`	`io_uring`	C + handler, GIL per request
`uringpy-app-batch`	`io_uring`	C + handler, GIL per batch
`c-epoll-app`	`epoll`	C + handler, GIL per request
`c-epoll-app-batch`	`epoll`	C + handler, GIL per batch
`asyncio-app`	`epoll`	Python, streams API + handler
`asyncio-proto-app`	`epoll`	Python, Protocol API + handler
`uvloop-proto-app`	libuv	C loop, Protocol API + handler

quest's processor time now falls under the lock. In terms of the model, the first control lowers $\hat{g}$ without touching the interface or the system calls, and the second raises f without raising $\hat{g}$. Neither is a perfectly clean single-factor change, and Section 5.5 says where they fall short. The cleanest comparison between two engines is between the reactors that call a Python handler: `c-epoll-app` and `c-epoll-app-batch` make the same system calls and run the same Python per request, and differ in taking the GIL once per request or once per batch.

5.3 Metrics

For a configuration with n runs and throughputs $x_1, \ldots, x_n$ (requests per second over the measured interval) we report the mean $\bar{x}$ together with the half-width e of its 95% confidence interval,

$$e = t_{0.975,\,n-1}\,\frac{s}{\sqrt{n}}, \qquad s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2, \quad (3)$$

where t is the Student-t quantile (2.776 for $n = 5$). Scaling at N workers is the ratio to the one-worker configuration of the same engine and mode, and its half-width follows from first-order error propagation:

$$S_N = \frac{\bar{x}_N}{\bar{x}_1}, \qquad e_S = t\,S_N\sqrt{\frac{s_N^2}{n_N\,\bar{x}_N^2} + \frac{s_1^2}{n_1\,\bar{x}_1^2}}, \quad (4)$$

with t taken at $\min(n_N, n_1) - 1$ degrees of freedom. Any other ratio of two configurations is computed and reported in the same way.

We sample processor usage from `/proc/stat` on both machines immediately before and after each measured interval. If u is the busy share of the server's four cores, then $C = 4u$ is the number of busy cores. We report throughput per busy core, $\bar{x}/C$, next to S_N, because even a one-worker server uses other cores for kernel network processing and S_N by itself understates per-core scaling. We also give this quantity relative to one worker, $E_N = (\bar{x}_N/C_N)/(\bar{x}_1/C_1)$, so that $S_N = E_N\,C_N/C_1$. A configuration is flagged if the client's busy share exceeds 85%, since the load generator may then be the limiting factor.

The remaining metrics come from the servers' own counters, summed over a configuration's runs and then divided. They are system calls per request (the I/O calls the loop issues, not the `futex` calls the interpreter makes when the GIL is contended), completions per `io_uring_enter`, and $\hat{g}$, the upper limit on GIL handoffs per request (Section 3). For the C reactors that call a handler, $\hat{g}$ is the counted number of GIL acquisitions. For the Python loops it is the number of calls that release the GIL: every system call that `py-epoll` makes through CPython's socket and `select` modules, and every call into the runtime that waits or submits for the other three. In application mode with timing enabled we also report the GIL hold time t_p and wait time w per handler call. Timing is enabled in the handler sweep and the cap sweep (Tables 9 and 10) and disabled everywhere else. It costs a few percent: at four worker threads the timed per-request reactor delivers 0.96 of the untimed one. Finally, every server process reports its voluntary context switches from `getrusage`. A thread that goes to sleep waiting for the GIL, or for I/O, counts as one. For our own engines we divide this by the number of requests the server counted. `asyncio` and `uvloop` do not count requests, so for them we divide by the requests that `wrk` counted, scaled from the measured interval to the server's lifetime. Where both methods can be applied they agree to within 4%. With 400 connections, workers that each have their own GIL hardly ever sleep: no process-mode configuration of Tables 4, 7 and 8 makes more than 0.004 voluntary switches per request. Under that load the switches we see in thread mode are therefore waits for the GIL and not waits for I/O. Latency percentiles are taken from `wrk` and are used only to compare configurations under the same load (Section 2).

5.4 How the hypothesis changed

The claim we test is not the one we started with, and that affects how each result should be read.

We began with a broader hypothesis: for a Python `io_uring` server, where the per-event loop runs, in C or in Python, matters more than which system-call interface it uses. Its criterion was that moving the loop from Python to C should change throughput far more than switching between `epoll` and `io_uring`, and that capping the completions processed per system call should change it only slightly. We wrote this down while the first measurements were already running, so it was not pre-registered. Those measurements seemed to support part of it. The Python loop on `io_uring` did not scale across threads, and we concluded that a Python loop could not scale on either

interface. That was wrong. Our loop was holding the GIL during its submitting system call, which nothing in the design required, and once the GIL was released around that call the same loop scaled. We kept the faulty variant as the control `py-uring-held`, and the earlier runs remain in the repository, marked as superseded.

The corrected 2×2 design does not meet the criterion. On one worker the language of the loop is worth 1.08–1.17× and the interface 1.15–1.25×, so neither dominates. Capping the batch changes little at 16 completions or more and costs up to a fifth of the throughput below that (Section 5.5). We therefore do not claim that the system call is unimportant or that the loop has to be in C. The data pointed instead to the statement of Section 3, that thread scaling depends on how much of a request runs under the GIL and on how often the GIL changes hands. Since we arrived at this statement after looking at those data, the 2×2 results are the observations that led to it and not a test of it.

To test it we designed three further experiments and ran them afterwards: the control `py-epoll-batch`, the `epoll` reactor with GIL batching, and a sweep over the number of requests served per GIL acquisition. None of the three had been run on the testbed before. For the first we wrote the prediction, and what would refute it, into the repository's description of the experiments, in the commit at which the main run was then made; the result folders carry the hash of that commit. If handoffs decide thread scaling, `py-epoll-batch` should scale like `py-uring` even though it uses `epoll`; if it scales like `py-epoll`, the explanation is wrong. The other two experiments are listed there as open questions. For the supplementary run we wrote down what we expected of each measurement in the same way, in the commit it was run from. Those expectations were weaker, a preserved ordering and a steady fall, and we report them with the results.

5.5 Results: loops without a handler

All tables and figures, and every number in the text that comes from the benchmark runs, are generated from the raw runs by a script in the repository (`benchmarks/make_tables.py`). The experiments reported here comprise 2190 runs. 50 of them, the 1600-connection runs of the main run, are left out of every table and figure, because that number of connections exceeded the limit on open files in the server's container; they were repeated in the supplementary run. The data set is the remaining 2140 runs of 428 configurations. No run in it failed, returned a non-2xx response or had a handler error (the script counts 0, 0 and 0), and the hypervisor reported no steal time. Averaged over the runs of a configuration, the client's busy share stayed at or below 78%; 2 individual runs went above 85%, the highest at 88%. 133 runs reported socket errors, for at most 0.37% of a run's requests. Section 7 says which, and describes the runs that are not part of the data set.

The 2×2 design. Table 4 shows the four cells, and Table 5 gives the effect of each factor with the other held fixed. The counters confirm that the design varies what it is supposed to. At four workers the two `epoll` loops make 2.014 and 2.012 system calls per request, one `recv` and one `send`, while the two `io_uring` loops make 0.022 and 0.042.

On one worker both factors have an effect, and neither is dominant. Switching from `epoll` to `io_uring` raises throughput by a factor of 1.15 ± 0.02 for the C loop and 1.25 ± 0.05 for the Python loop. Moving the loop from Python to C raises it by 1.08 ± 0.04 on `io_uring` and by 1.17 ± 0.03 on `epoll`. A full event loop would pay more than this second pair suggests, because our Python loops are so bare. `asyncio`'s streams API, which also runs on `epoll`, reaches 67.6 thousand requests per second on one worker (Table 7), compared with 107.4 for `py-epoll`.

What scaling can mean on this machine needs a word first. Four worker processes, which share no lock, keep all four cores busy, part of that with kernel network processing, and scale by between 2.29 and 2.49 (column S_4^{proc}). The thread configurations that scale are in the same position ($C_4 = 3.81$–3.92). For these loops a thread scaling of about 2.3 therefore means "as good as processes on this machine" and not 2.3 out of an ideal 4, and differences between configurations that reach this ceiling are compressed.

Four workers gave us a result we had not expected. Both C loops scale, to $S_4 = 2.28 \pm 0.03$ on `io_uring` and 2.36 ± 0.03 on `epoll`. But so does the Python loop on `io_uring`, with $S_4 = 2.19 \pm 0.07$ and 293.8 thousand requests per second. That is 0.89 of the C loop on the same interface and 0.99 of the C loop on `epoll`. The Python loop on `epoll` behaves quite differently. It gains a little at two workers (1.30 ± 0.05) and then drops below the throughput of a single worker at four (0.62 ± 0.02). The difference is already there with two workers, where more than one core is still idle: the other three loops gain between 1.58× and 1.64×. So the placement of the loop is not what decides scaling. On `io_uring`, moving the loop from Python to C is worth 1.08× on one worker and 1.12× on four. Taken at face value, the table says that the interface is what separates the two Python loops, by a factor of 1.25 on one worker and 4.40 on four. The saved system calls cannot be the reason, because `c-epoll` makes two per request and scales as well as anything. The design cannot tell us more than that, however, because of the confound described above: between `py-epoll` and `py-uring` the GIL releases change along with the interface.

The release control. This is what `py-epoll-batch` is for. It uses `epoll` and makes 2.013 system calls per request, the same as `py-epoll` (2.012). The intended difference is that all the `recv` and `send` calls of a pass are issued by one C call that releases the GIL once. At four workers the loop then releases the GIL 0.026 times per request, where `py-epoll` releases it 2.012

Table 4: The 2×2 design and two controls (worker threads, GIL build, 13-byte body): throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$) at N workers; scaling of worker threads S_4, of worker processes S_4^{proc} and of worker threads on the free-threaded build S_4^{ft}; and, at four worker threads, per request: measured system calls (sys), counted GIL releases ($\hat{g}$, an upper limit on hand-offs) and voluntary context switches (c.sw.); busy server cores C_4; and 99th-percentile latency in ms.

Engine	Loop	N=1	N=2	N=4	S_4	S_4^{proc}	S_4^{ft}	sys	$\hat{g}$	c.sw.	C_4	p99
`uringpy`	C	144.3 ± 1.9	237.2 ± 4.5	328.7 ± 0.7	2.28 ± 0.03	2.30 ± 0.05	2.30 ± 0.06	0.022	0	0.001	3.91	1.7
`c-epoll`	C	125.6 ± 1.4	203.4 ± 4.8	296.9 ± 0.6	2.36 ± 0.03	2.35 ± 0.04	2.38 ± 0.05	2.014	0	0.001	3.92	2.8
`py-uring`	Python	133.9 ± 4.4	211.6 ± 3.9	293.8 ± 2.3	2.19 ± 0.07	2.36 ± 0.05	2.33 ± 0.03	0.042	0.042	0.013	3.81	1.9
`py-epoll`	Python	107.4 ± 2.3	139.8 ± 3.9	66.7 ± 1.7	0.62 ± 0.02	2.49 ± 0.05	2.40 ± 0.08	2.012	2.012	2.048	2.48	11.6
`py-epoll-batch`[a]	Python	124.3 ± 2.8	199.2 ± 6.0	292.0 ± 1.1	2.35 ± 0.05	2.36 ± 0.07	2.33 ± 0.03	2.013	0.026	0.003	3.90	2.8
`py-uring-held`[b]	Python	135.4 ± 6.6	129.2 ± 3.2	127.6 ± 0.4	0.94 ± 0.05	2.29 ± 0.06	2.30 ± 0.06	0.039	0.020	0.021	1.51	667.0

The upper four rows are the design; the lower rows are controls, each aimed at one respect in which two cells differ (see the text). [a] `py-epoll` with the `recv` and `send` calls of all ready sockets made by one C call that releases the GIL once. [b] `py-uring` holding the GIL during the submitting system call.

Table 5: Effect of each factor with the other held fixed, as a ratio of mean throughputs (± 95% half-width) from Table 4. Interface: `io_uring` over `epoll`. Loop: C over Python.

Effect	N=1	N=2	N=4
Interface, loop in C	1.15 ± 0.02	1.17 ± 0.04	1.11 ± <0.01
Interface, loop in Python	1.25 ± 0.05	1.51 ± 0.05	4.40 ± 0.12
Loop, on `io_uring`	1.08 ± 0.04	1.12 ± 0.03	1.12 ± 0.01
Loop, on `epoll`	1.17 ± 0.03	1.46 ± 0.05	4.45 ± 0.12
GIL releases (`epoll`)[a]	1.16 ± 0.04	1.43 ± 0.06	4.38 ± 0.11
Interface (batched)[b]	1.08 ± 0.04	1.06 ± 0.04	1.01 ± 0.01

[a] `py-epoll-batch` over `py-epoll`: the same interface and the same system calls, with the GIL released once for all the `recv` and `send` calls of a pass instead of once per call. The batched loop also runs less Python per request. [b] `py-uring` over `py-epoll-batch`: both release the GIL about twice per pass, but `py-uring` runs more Python per event, so this ratio is not the effect of the interface alone.

times. Four threads scale by 2.35 ± 0.05 in place of 0.62 ± 0.02. They deliver 292.0 thousand requests per second, which is 4.38 ± 0.11 times what `py-epoll` manages and 0.99±0.01 of what `py-uring` does. As predicted in Section 5.4, `py-epoll-batch` scales like `py-uring` and not like `py-epoll`. The two `epoll` loops make the same system calls, so the system calls cannot be what separates them.

The context switches are consistent with this reading. A single `py-epoll` worker makes almost no voluntary context switches, fewer than one per thousand requests. Four workers make 2.048 per request, roughly one for every release of the lock, as if a worker had to sleep almost every time it lets go of the GIL. The batched loop makes 0.003 per request and `py-uring` makes 0.013. The loops themselves are not at fault. Run as four processes, `py-epoll` scales by 2.49 ± 0.05, the most of the six (Table 4), so the loss comes from sharing the lock. The free-threaded build confirms this from the other side. With no lock to share, all six loops scale alike as threads, by between 2.30 and 2.40 (column S_4^{ft}).

The control is not a single-factor change, however. `BatchIO` also takes the `recv` and `send` calls out of CPython's socket objects, so the batched loop runs less Python per request. We did not instrument the lock in the Python loops, but the one-worker throughputs give an estimate. Compared with the C loop on the same interface, a request costs 1.35 ± 0.22 μs more in `py-epoll`, 0.54±0.26 μs more in `py-uring` and 0.08±0.20 μs more in `py-epoll-batch`, the last not distinguishable from zero. Taking that extra time as the time under the GIL puts f at about 0.145, 0.072 and 0.010. In terms of time under the lock the batched loop is thus nearly a C loop. The control therefore lowers f as well as $\hat{g}$, and it is 1.16±0.04 times faster on one worker. Two observations show that the lower f is not what restores scaling. First, time under the lock could not have been the limit for `py-epoll`: with $f \approx 0.145$ the bound (1) allows $1/f \approx 6.9$, more than the machine has cores, and no value of f explains a result below one worker. Second, the reactors with a handler, discussed below, make the same change of $\hat{g}$ with the Python per request held fixed, and show the same effect. For the same reason the ratio of `py-uring` to `py-epoll-batch` (1.08 on one worker) is not the effect of the interface alone, since `py-uring` runs more Python per event, but it is closer to it than the ratio of `py-uring` to `py-epoll` (1.25), which also contains the cost of releasing the GIL at every call.

The hold-time control. Counting hand-offs does not tell the whole story, and (1) says it should not. `py-uring-held` gives up the GIL only 0.020 times per request, which is less often than `py-uring`. It keeps the lock while submitting, though, and the kernel does the queued sends inside that call. We did not time the lock in this loop, but most of a request's processor time must then fall under the GIL. Four workers end up delivering slightly less than one (0.94 ± 0.05), with only 1.51 of the four cores busy. `wrk` reports a 99th-percentile latency of 667.0 ms against a median of 0.9 ms, so some connections are served quickly while others starve. Releasing the GIL around that single call accounts for the whole difference between 0.94× and 2.19×. On the free-threaded build the same loop scales by 2.30×. Few hand-offs are thus

Table 6: Batch cap (`uringpy`, one worker): completions processed per `io_uring_enter` limited to the cap. Throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$), ratio to the uncapped run, measured completions per call and system calls per request.

Cap	Throughput	vs. none	compl./call	sys/req
1	116.4 ± 1.1	0.81 ± 0.02	1.0	2.001
4	133.2 ± 2.3	0.93 ± 0.03	4.0	0.501
16	138.3 ± 3.2	0.97 ± 0.03	16.0	0.126
64	143.1 ± 7.0	1.00 ± 0.05	63.3	0.032
256	141.2 ± 4.0	0.99 ± 0.04	234.6	0.009
none	143.0 ± 3.2	1	357.7	0.006

necessary for threads to scale under the GIL, but they are not sufficient, because what runs under the lock still counts.

Batching of system calls. Table 6 isolates the effect of batching system calls, using one worker of the C reactor. When we cap it at one completion per call, `uringpy` makes 2.001 system calls per request, about as many as an `epoll` loop, and delivers 0.81 ± 0.02 of its uncapped throughput. A cap of 4 gives 0.93 ± 0.03 and a cap of 16 gives 0.97 ± 0.03, even though system calls per request keep falling beyond that point, from 0.126 down to 0.006. For a single worker, then, the system calls saved by batching are worth roughly a fifth of its throughput, and almost all of that has been collected by the time a batch holds 16 completions. In practice batches are much larger. An uncapped single worker sees 358 completions per call, and each of four workers, which share the 400 connections, sees 91. With the cap at one, `io_uring` delivers 116.4 thousand requests per second, against 125.6 for the `epoll` loop (0.93 of it; the two figures come from different experiments). Without batching, in other words, the completion interface has no advantage on this workload and a small deficit.

5.6 Results: the runtime against `asyncio` and `uvloop`

Scaling across cores. Fig. 3 and Table 7 compare `uringpy` with `asyncio` and `uvloop` on the transport path.

First, `uringpy` worker threads perform like worker processes. At four workers the ratio between the two is $1.00 \pm <0.01$, so on this path one interpreter with four threads delivers what four interpreters deliver. A path that runs no Python should behave this way, and the measurement shows that nothing else in the runtime or the interpreter binds before this machine's ceiling does.

Second, its scaling, $S_4 = 2.28 \pm 0.04$, is not limited by the GIL, since its processes reach the same value. A single worker already keeps $C_1 = 1.19$ cores busy, because some of the kernel's network processing for its connections runs on other cores, and four workers keep 3.91 of the four cores busy. Per busy core, throughput at four workers is $E_4 = 0.69$ of what it is at one. The baselines show the same loss in process mode, where no lock is shared at all: their E_4 values are 0.73, 0.73, 0.75 and 0.72, and they scale by 2.66, 2.55, 2.70 and 2.51. The loss therefore belongs to the testbed, where four cores are shared between the workers and the kernel's network stack. Process mode marks what the hardware allows (Section 3), and thread-mode `uringpy` reaches it.

Third, every baseline delivers less with four worker threads than with one. `asyncio` delivers 0.51× and 0.48× of one worker through its two APIs, and `uvloop` delivers 0.60× and 0.36×, with between 1.59 and 2.18 of the four cores busy. This is the same pattern we saw for `py-epoll` in Table 4, and the context switches point the same way. The `asyncio-proto` threads make 3.224 voluntary context switches per request at four workers and `uvloop-proto` threads 3.683. With one worker both make fewer than one per thousand requests. Having the loop in C is not enough to avoid this. `uvloop`'s loop is libuv, but it calls back into Python for every event, and its threads lose throughput just as `asyncio`'s do. On one worker `uvloop` is only 1.06× and 1.04× faster than `asyncio` through the two APIs. One small request per receive leaves a faster loop little to save, and we did not tune either loop.

In absolute terms, one `uringpy` worker is 1.49× faster than one `uvloop-proto` worker, 1.55× faster than `asyncio-proto` and 2.14× faster than `asyncio`. At four workers the fair comparison is with the baselines run as processes, which is how they have to be run under the GIL. Against those, `uringpy` threads are 1.33× faster than `uvloop-proto`, 1.41× faster than `asyncio-proto` and 1.83× faster than `asyncio`. This workload flatters the runtime about as much as any could, since the baselines run Python for every event and `uringpy` runs none.

Free-threaded build. The right-hand panel of Fig. 3 and the lower part of Table 7 repeat the experiment without the GIL. `asyncio` threads now scale, by 2.64 ± 0.09 and 2.48 ± 0.05, and reach 0.97 and 0.98 of the throughput of the same engines in process mode. The collapse on the GIL build therefore requires the GIL: the same code scales once the lock is removed. On one worker the free-threaded build runs `asyncio` at 0.94 and 0.96 of its speed on the GIL build. `uringpy` hardly depends on the build, as one would expect of a loop that does not enter the interpreter: its throughput at four workers on the two builds differs by a factor of $0.99 \pm <0.01$. What is left of its advantage is a difference in raw speed: 1.59× over `asyncio-proto` and 2.25× over `asyncio` on one worker, and 1.46× and 1.94× on four. On the free-threaded build, then, the runtime contributes speed per worker and no longer contributes scaling.

5.7 Results: a Python handler and GIL batching

A Python handler: the GIL taken per request. In Table 8 every request calls a Python function that parses the request line, computes a checksum of the path and builds the response. On one worker the handler

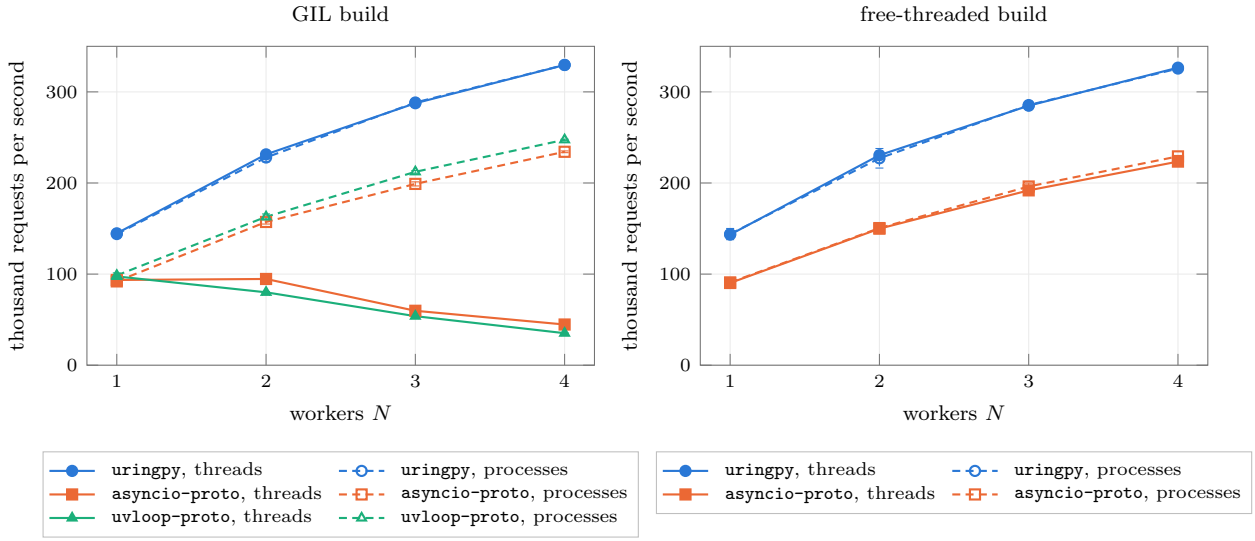


Figure 3: Transport path: throughput against the number of workers, as threads of one interpreter (solid lines, filled markers) and as one process per worker (dashed lines, open markers). Bars are 95% half-widths; most are smaller than the markers. `uvloop` is not available on the free-threaded build.

leaves `uringpy` with 0.87 of the transport-only throughput in Table 7. When the GIL is taken once per request (`uringpy-app`), worker threads stop scaling after the second one: $S_2 = 1.55 \pm 0.05$ and $S_4 = 1.61 \pm 0.07$, with 3.28 of the four cores busy. Worker processes scale as they do on the transport path ($S_4 = 2.35 \pm 0.06$), so at four workers the threads deliver 0.68 ± 0.02 of what the processes do.

The bound (1) does not explain this, but the handler sweep in Table 9 does. (That table was measured with GIL timing on, and its S_4 for the same configuration is 1.56.) On one worker the handler holds the GIL for $t_p = 1.18$ μs per request, which gives $f = 0.15$. The bound $1/f$ would allow more scaling than four cores can provide. At four threads, however, a request holds the GIL for 2.12 μs and waits 7.79 μs to get it. The GIL utilisation is only $U_4 = 0.41$. For the rest of each second the lock is not running a handler at all, and yet workers are waiting for it. This waiting is the gap between threads and processes. Per worker, a request takes 20.7 μs in thread mode and 13.6 μs in process mode.

If we assume that every acquisition is a hand-off ($g = \hat{g} = 1$) and that (2) holds with equality, the cost of a hand-off is the residual $\tau = 1/\bar{x}_4 - t_{p,4}$. This is a difference of two measured numbers, and it is precise to better than a microsecond only for the 7 smaller handler sizes of Table 9 (up to 300 iterations). For those it comes out between 2.9 and 5.4 μs. If the whole residual is attributed to hand-offs, each thus costs several microseconds, and when the handler itself takes only a microsecond or two the lock does little else. The spread also illustrates the limitation noted in Section 3: no single value of τ fits all sizes, and since τ is computed as a residual it cannot test (2).

How many acquisitions turn into real hand-offs depends on the handler's length. As the handler grows from the unmodified one to 20 added iterations (t_p from 1.18 to 3.06 μs), the per-request reactor's voluntary context switches per request at four threads rise step by step from 0.407 to 1.337, and its scaling falls from 1.56 to 1.02. Once a handler runs long enough for a waiting thread to be woken in the meantime, the reactor makes one switch per acquisition or more, as it would if every acquisition were a real hand-off. For longer handlers scaling stays close to one, between 0.95 and 1.14.

GIL batching: the GIL taken per batch. If the reactor takes the GIL once for all the requests it finds in a pass over the completion queue, the Python run per request stays the same and $\hat{g}$ drops from 1 to 0.021. In Table 8 the threads then scale by 2.18 ± 0.05, up from 1.61 ± 0.07. Four threads deliver 279.5 thousand requests per second, compared with 201.8 before (1.39 ± 0.05). That is 0.93 ± 0.02 of what four processes deliver, and a single worker is not slowed down (1.02 ± 0.03). In Table 9 the wait per request at four threads falls from 7.79 to 1.65 μs; a batch still waits about 79 μs for the lock, but only once. The busy cores rise from 3.28 to 3.72. The gain belongs to four workers. With two, where cores are still idle, batching is worth 1.06 ± 0.02 for this reactor. Batching delays the first response of a batch until the last handler of that batch has run, so one might expect latency to suffer. This load cannot show whether it does. With a fixed number of connections that each wait for their reply, mean latency is the number of connections divided by the throughput, so the fall of the median from 1.9 to 1.4 ms only restates the gain in throughput. The 99th percentile falls further, from 3.4 to 1.9 ms, so the spread between requests narrows. What batching costs in latency would show below saturation, which we did not measure.

Table 10 varies $\hat{g}$ in steps while leaving everything else

Table 7: Transport path, 13-byte body: throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$) at N workers; scaling $S_4 = \bar{x}_4/\bar{x}_1$; busy server cores C_N; throughput per busy core at four workers relative to one, $E_4 = (\bar{x}_4/C_4)/(\bar{x}_1/C_1)$; 99th-percentile latency at four workers.

Build	Engine	Workers	N=1	N=2	N=3	N=4	S_4	C_1	C_4	E_4	p99 (ms)
GIL	`uringpy`	thread	144.8 ± 2.3	231.4 ± 3.9	287.5 ± 4.4	329.5 ± 0.6	2.28 ± 0.04	1.19	3.91	0.69	1.6
GIL	`uringpy`	process	144.0 ± 4.3	228.0 ± 4.6	288.3 ± 3.8	329.6 ± 0.6	2.29 ± 0.07	1.19	3.92	0.70	1.6
GIL	`asyncio`	thread	67.6 ± 0.7	47.9 ± 0.8	35.5 ± 0.4	34.6 ± 0.6	0.51 ± 0.01	1.08	1.76	0.31	15.3
GIL	`asyncio`	process	67.8 ± 1.0	117.9 ± 1.3	152.4 ± 2.6	180.3 ± 0.8	2.66 ± 0.04	1.08	3.91	0.73	2.6
GIL	`asyncio-proto`	thread	93.6 ± 1.2	94.7 ± 1.8	59.7 ± 0.2	44.7 ± 0.3	0.48 ± 0.01	1.12	2.18	0.25	16.8
GIL	`asyncio-proto`	process	92.0 ± 1.1	157.2 ± 2.3	199.0 ± 2.2	234.3 ± 1.0	2.55 ± 0.03	1.11	3.90	0.73	3.5
GIL	`uvloop`	thread	71.4 ± 1.0	67.8 ± 1.1	52.7 ± 0.7	42.8 ± 0.5	0.60 ± 0.01	1.09	1.67	0.39	20.3
GIL	`uvloop`	process	72.2 ± 1.3	126.5 ± 1.1	164.6 ± 1.7	194.8 ± 1.6	2.70 ± 0.05	1.09	3.92	0.75	4.5
GIL	`uvloop-proto`	thread	97.5 ± 2.3	80.0 ± 0.9	53.8 ± 0.2	35.1 ± 0.2	0.36 ± 0.01	1.13	1.59	0.26	26.8
GIL	`uvloop-proto`	process	98.6 ± 2.8	162.9 ± 3.2	212.3 ± 1.4	247.4 ± 0.7	2.51 ± 0.07	1.13	3.92	0.72	3.4
no GIL	`uringpy`	thread	143.4 ± 3.3	230.5 ± 4.4	284.9 ± 3.9	326.6 ± 1.5	2.28 ± 0.05	1.19	3.91	0.69	1.7
no GIL	`uringpy`	process	143.9 ± 6.1	227.1 ± 10.7	285.5 ± 4.3	325.6 ± 2.2	2.26 ± 0.10	1.20	3.92	0.69	1.8
no GIL	`asyncio`	thread	63.6 ± 1.9	107.3 ± 1.1	140.9 ± 1.2	168.1 ± 2.5	2.64 ± 0.09	1.08	3.93	0.72	2.9
no GIL	`asyncio`	process	64.4 ± 1.1	110.0 ± 2.2	145.3 ± 2.0	173.2 ± 1.4	2.69 ± 0.05	1.08	3.91	0.74	2.8
no GIL	`asyncio-proto`	thread	90.2 ± 1.8	149.9 ± 3.7	191.9 ± 1.8	223.5 ± 0.8	2.48 ± 0.05	1.12	3.92	0.71	3.8
no GIL	`asyncio-proto`	process	90.9 ± 1.0	150.5 ± 2.6	196.2 ± 3.7	229.2 ± 1.0	2.52 ± 0.03	1.12	3.93	0.72	3.6

"no GIL" is the free-threaded build; `uvloop` is not installed in its image.

alone, by capping the number of requests handled under one acquisition. These are the connected points in Fig. 4. With a cap of one, the batching code takes the GIL for every request, just as the plain reactor does, and it scales the same way (1.55 ± 0.06 against 1.57 ± 0.03). Any cost of the batching code path itself is below what we can resolve. As the cap goes up through 2, 4, 8 and 16, the acquisitions per request fall to 0.505, 0.258, 0.134 and 0.072, and scaling rises to 1.78, 1.93, 2.04 and 2.12. With a cap of 64 and with no cap it is 2.20±0.04 and 2.17±0.03, which cannot be told apart. The wait for the lock falls in step, from 8.01 to 1.55 μs per request, and so do the context switches, from 0.431 to 0.013 per request. Up to a cap of 16 every step moves in the direction (2) predicts when f is fixed, and most of the gain is collected early: a cap of 8 already gives $S_4 = 2.04$.

GIL batching does not depend on `io_uring`, and the `epoll` reactor gives the cleanest of our comparisons between two engines. Its two variants make the same system calls and execute the same handler, so the Python run per request is the same in both. They differ in when the GIL is taken, and with it in the order of the work within a pass: the batched variant reads from every ready socket before it runs the handlers. On one worker that makes it 1.11 ± 0.03 times faster. The `epoll` reactor scales by 0.70 ± 0.01 when it takes the GIL per request, and by 2.33 ± 0.05 when it takes it once per pass over the ready sockets, a gain of 3.73 ± 0.04 in throughput at four workers, which brings its threads to 0.95±0.01 of its processes. For this reactor the gain is already 1.48±0.03 at two workers. With batching enabled in both, the `io_uring` reactor is 1.11 ± 0.03 times faster than the `epoll` reactor on one worker and 1.04 ± 0.02 times faster on four. As on the transport path, the interface adds some speed per worker, while the scaling comes from how the GIL is handled.

Opportunities and actual hand-offs. The two reactors that take the GIL per request have the same count, $\hat{g} = 1$, and yet they behave very differently: four threads scale by 1.61 on `io_uring` and by 0.70 on `epoll`. The context switches point to where the difference lies. Per request, the `epoll` reactor makes 1.438 voluntary switches and the `io_uring` reactor 0.363. The `epoll` reactor thus sleeps more than once for every acquisition, and the `io_uring` reactor about once for every three. We did not trace the lock, so we cannot say for certain why, but the structure of the two loops suggests a reason. The `io_uring` reactor works through a batch of completions without a system call. Only a few instructions separate one acquisition from the next, and the thread often has the lock back before a waiter has been woken. The `epoll` reactor makes a `send` and a `recv` system call between two acquisitions, which gives a waiter time to take the lock. This is why we treat the count $\hat{g}$ as an upper limit (Section 3): how many acquisitions become hand-offs depends on what the loop does between them. It also means the interface is not irrelevant to scaling when the GIL is taken per request. Per-request locking is 2.80× faster on `io_uring` than on `epoll` at four threads, against 1.21× on one. The effect again runs through the lock and not through the cost of the system calls, since as four processes the same two reactors differ by only 1.15×.

All configurations on one plot. Because a count of releases does not tell us how many hand-offs took place, Fig. 4 plots thread scaling against voluntary context switches per request. These are available for every engine, including `asyncio` and `uvloop`, whose GIL releases we cannot count. The picture is two clusters and, between them, the sweep of one engine. The 7 configurations that make at most 0.015 switches per request scale by between 2.17 and 2.36. These are the two C loops, the two Python loops that release the GIL per pass, and the reactors with GIL batching, uncapped or

Table 8: Application-handler workload: every request calls a Python handler. Throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$), scaling, and at four workers: GIL acquisitions per request ($\hat{g}$) where the reactor counts them and the workers share one GIL, voluntary context switches per request (c.sw.), busy server cores and 99th-percentile latency.

Build	Engine	Workers	N=1	N=2	N=4	S_2	S_4	$\hat{g}$	c.sw.	C_4	p99 (ms)
GIL	`uringpy-app`	thread	125.5 ± 3.7	194.7 ± 2.1	201.8 ± 6.3	1.55 ± 0.05	1.61 ± 0.07	1.000	0.363	3.28	3.4
GIL	`uringpy-app`	process	125.4 ± 2.7	206.6 ± 5.8	295.1 ± 4.0	1.65 ± 0.06	2.35 ± 0.06	–	0.001	3.92	1.8
GIL	`uringpy-app-batch`	thread	128.0 ± 1.9	205.6 ± 1.9	279.5 ± 4.5	1.61 ± 0.03	2.18 ± 0.05	0.021	0.014	3.72	1.9
GIL	`uringpy-app-batch`	process	128.9 ± 3.3	209.2 ± 3.9	299.2 ± 2.0	1.62 ± 0.05	2.32 ± 0.06	–	0.001	3.92	1.7
GIL	`c-epoll-app`	thread	103.3 ± 1.7	127.2 ± 1.0	72.0 ± 0.5	1.23 ± 0.02	0.70 ± 0.01	1.000	1.438	2.27	10.9
GIL	`c-epoll-app`	process	104.7 ± 1.3	171.4 ± 5.3	256.2 ± 2.1	1.64 ± 0.05	2.45 ± 0.04	–	0.001	3.91	3.1
GIL	`c-epoll-app-batch`	thread	115.1 ± 2.4	188.6 ± 3.2	268.4 ± 2.1	1.64 ± 0.04	2.33 ± 0.05	0.013	0.007	3.79	3.0
GIL	`c-epoll-app-batch`	process	117.3 ± 1.7	191.6 ± 3.4	282.2 ± 1.0	1.63 ± 0.04	2.41 ± 0.04	–	0.001	3.91	2.9
GIL	`asyncio-app`	thread	59.8 ± 1.2	40.1 ± 0.5	31.0 ± 0.4	0.67 ± 0.02	0.52 ± 0.01	–	2.860*	1.68	16.6
GIL	`asyncio-app`	process	59.9 ± 0.6	104.2 ± 2.1	157.5 ± 10.3	1.74 ± 0.04	2.63 ± 0.17†	–	0.000*	3.92	4.0
GIL	`asyncio-proto-app`	thread	79.6 ± 1.4	64.5 ± 1.4	38.6 ± 0.2	0.81 ± 0.02	0.49 ± 0.01	–	3.029*	1.92	18.3
GIL	`asyncio-proto-app`	process	80.4 ± 1.1	136.2 ± 2.7	177.8 ± 42.0	1.70 ± 0.04	2.21 ± 0.52†	–	0.000*	3.92	5.7
GIL	`uvloop-proto-app`	thread	85.8 ± 1.3	68.4 ± 0.4	29.8 ± 0.3	0.80 ± 0.01	0.35 ± 0.01	–	3.826*	1.45	32.6
GIL	`uvloop-proto-app`	process	85.8 ± 0.5	142.1 ± 1.9	218.7 ± 4.8	1.66 ± 0.02	2.55 ± 0.06	–	0.004*	3.88	4.2
no GIL	`uringpy-app`	thread	122.7 ± 2.1	200.9 ± 4.6	290.3 ± 1.0	1.64 ± 0.05	2.37 ± 0.04	–	0.001	3.92	1.9
no GIL	`uringpy-app`	process	121.7 ± 3.8	200.3 ± 5.6	289.6 ± 4.7	1.65 ± 0.07	2.38 ± 0.08	–	0.002	3.90	2.0
no GIL	`uringpy-app-batch`	thread	126.9 ± 2.6	202.6 ± 2.7	291.4 ± 4.8	1.60 ± 0.04	2.30 ± 0.06	–	0.001	3.91	1.9
no GIL	`uringpy-app-batch`	process	125.7 ± 2.7	205.0 ± 6.8	294.2 ± 1.8	1.63 ± 0.06	2.34 ± 0.05	–	0.001	3.91	1.8
no GIL	`c-epoll-app`	thread	102.1 ± 2.0	164.2 ± 3.2	248.1 ± 1.1	1.61 ± 0.04	2.43 ± 0.05	–	0.001	3.92	3.3
no GIL	`c-epoll-app`	process	102.8 ± 2.3	167.3 ± 3.8	253.6 ± 0.8	1.63 ± 0.05	2.47 ± 0.06	–	0.001	3.92	3.3
no GIL	`c-epoll-app-batch`	thread	114.3 ± 2.3	187.9 ± 2.2	275.9 ± 0.6	1.64 ± 0.04	2.41 ± 0.05	–	0.001	3.91	2.8
no GIL	`c-epoll-app-batch`	process	114.6 ± 2.3	188.9 ± 1.5	279.1 ± 0.6	1.65 ± 0.04	2.44 ± 0.05	–	0.001	3.91	2.9
no GIL	`asyncio-app`	thread	56.8 ± 1.1	95.5 ± 2.0	150.6 ± 1.0	1.68 ± 0.05	2.65 ± 0.05	–	0.000*	3.91	3.1
no GIL	`asyncio-app`	process	57.4 ± 1.0	98.8 ± 1.2	156.1 ± 1.0	1.72 ± 0.04	2.72 ± 0.05	–	0.000*	3.91	3.0
no GIL	`asyncio-proto-app`	thread	78.5 ± 1.1	128.2 ± 2.1	197.3 ± 1.4	1.63 ± 0.04	2.51 ± 0.04	–	0.001*	3.93	4.1
no GIL	`asyncio-proto-app`	process	78.5 ± 1.8	134.0 ± 3.0	203.2 ± 1.7	1.71 ± 0.05	2.59 ± 0.06	–	0.000*	3.92	4.1

"no GIL" is the free-threaded build. * These servers do not count their requests; the switches are divided by the request count of the client, scaled from the measured interval to the life of the server. † A configuration in this row had a coefficient of variation above 5%.

capped at 64. The 9 configurations that switch more than once per request all deliver less with four threads than with one ($S_4 \leq 0.70$): `py-epoll`, the `epoll` reactor with per-request locking, and every `asyncio` and `uvloop` configuration. Between the clusters lie the other 6 points of the cap sweep, along which scaling falls from 2.12 to 1.55 as the switches per request rise from 0.037 to 0.431. One configuration fits neither cluster: `py-uring-held` switches rarely and does not scale, because it keeps the lock through the kernel's work.

The figure is descriptive. The boundaries quoted above were chosen after seeing the data. It is a correlation across engines, and both of its axes contain throughput, since the switches are divided by the requests served, so part of the slope on the right is arithmetic. What it shows is the separation: the configurations that scale make about a hundred times fewer switches per request than those that collapse, or fewer still. The evidence for cause is in the interventions, which are the two arrows in the figure and the cap sweep. Their main effect, though for the Python loop not their only one, is on how often an engine gives up the GIL.

Baselines with the handler. Given the same handler, the baselines' threads drop to 0.52× (`asyncio-app`), 0.49× (`asyncio-proto-app`) and 0.35× (`uvloop-proto-app`) of one worker. The strongest baseline is `uvloop-proto-app` in process mode, which reaches 218.7 thousand requests per second on four workers. (The `asyncio-proto-app` processes drifted downwards over their five repetitions, which gives that configuration its wide interval in Table 8; no ratio we quote depends on it.) Measured against the `uvloop` processes, `uringpy-app` threads deliver 0.92 ± 0.04 when the GIL is taken per request and 1.28 ± 0.04 with GIL batching. Process against process, `uringpy-app` delivers 1.35 ± 0.04. On one worker `uringpy-app` is 1.46× faster than `uvloop-proto-app`, 1.58× faster than `asyncio-proto-app` and 2.10× faster than `asyncio-app`.

5.8 Results: limits

Longer handlers: where the lock itself is the limit. As the handler gets longer, f approaches 1 and the bound (1) starts to bind (Fig. 5a). For short handlers it is far above what the machine allows and says nothing. With 300 added iterations ($t_p = 30.46$ µs, $f = 0.81$) the bound is 1.23 and we measure $S_4 = 1.05 \pm 0.03$. With 3000 iterations the bound is 1.03 and the measurement is 0.97 ± 0.03. Batching helps for as long as hand-offs are what keeps the lock busy, and stops helping once the handler takes over that role. With 30 added iterations it raises S_4 from 1.01 to 1.64 and the GIL utilisation from 0.50 to 0.93. With 100 iterations it raises S_4 from 1.14 to 1.28. From 300 iterations upward it changes throughput

Table 9: Handler work swept (iterations of an interpreted loop added to every request), GIL build unless stated, with the GIL taken once per request (`uringpy-app`) or once per batch of requests (`uringpy-app-batch`). One-worker throughput $\bar{x}_1$ in thousands of requests per second (mean ± 95% half-width, $n = 5$); GIL hold time per request t_p at one worker; $f = t_p\,\bar{x}_1$; the bound $\min(4, 1/f)$ of (1); measured scaling of worker threads (S_2, S_4), of worker processes (S_4^{proc}) and of threads on the free-threaded build (S_4^{ft}); and, at four worker threads, GIL hold time $t_{p,4}$ and wait time w_4 per request, GIL acquisitions per request $\hat{g}$, and GIL utilisation $U_4 = \bar{x}_4\,t_{p,4}$.

GIL per	Work	$\bar{x}_1$	t_p (μs)	f	bound	S_2	S_4	S_4^{proc}	S_4^{ft}	$t_{p,4}$	w_4	$\hat{g}$	U_4
request	0	123.7 ± 5.4	1.18	0.15	4.00	1.55 ± 0.09	1.56 ± 0.08	2.41 ± 0.05	2.39 ± 0.08	2.12	7.79	1.000	0.41
request	5	115.7 ± 3.1	1.64	0.19	4.00	–	1.29 ± 0.04	2.38 ± 0.06	–	2.80	12.57	1.000	0.42
request	10	109.8 ± 1.7	2.13	0.23	4.00	–	1.15 ± 0.03	2.48 ± 0.05	–	3.44	16.67	1.000	0.43
request	20	99.4 ± 0.8	3.06	0.30	3.29	–	1.02 ± 0.02	2.51 ± 0.07	–	4.54	22.85	1.000	0.46
request	30	92.1 ± 1.0	3.96	0.36	2.74	1.42 ± 0.03	1.01 ± 0.02	2.65 ± 0.05	2.36 ± 0.06	5.32	25.73	1.000	0.50
request	100	54.8 ± 3.4	11.26	0.62	1.62	1.23 ± 0.08	1.14 ± 0.07	3.10 ± 0.09	1.72 ± 0.03	12.36	42.17	1.000	0.77
request	300	26.7 ± 0.4	30.46	0.81	1.23	1.06 ± 0.02	1.05 ± 0.03	3.52 ± 0.08	1.26 ± 0.03	32.86	101.65	1.000	0.92
request	1000	8.71 ± 0.20‡	105.20	0.92	1.09	1.06 ± 0.03‡	0.95 ± 0.03	3.90 ± 0.14	1.11 ± 0.04	115.83	356.59	1.000	0.95
request	3000	3.10 ± 0.07‡	313.75	0.97	1.03	0.92 ± 0.13†‡	0.97 ± 0.03‡	4.00 ± 0.12	1.03 ± 0.05‡	328.84	994.16	1.000	0.98
batch	0	128.6 ± 2.2	1.21	0.16	4.00	1.56 ± 0.03	2.16 ± 0.04	2.31 ± 0.05	–	2.39	1.65	0.021	0.66
batch	5	119.7 ± 3.0	1.71	0.20	4.00	–	2.10 ± 0.06	2.37 ± 0.05	–	3.27	2.88	0.021	0.82
batch	10	114.0 ± 2.2	2.14	0.24	4.00	–	2.00 ± 0.04	2.43 ± 0.03	–	3.85	4.62	0.021	0.88
batch	20	101.9 ± 1.1	3.11	0.32	3.16	–	1.77 ± 0.04	2.51 ± 0.05	–	5.06	9.06	0.021	0.91
batch	30	95.5 ± 0.7	4.04	0.39	2.59	1.57 ± 0.07	1.64 ± 0.02	2.58 ± 0.07	–	5.91	12.11	0.021	0.93
batch	100	57.3 ± 0.9	10.93	0.63	1.60	1.33 ± 0.04	1.28 ± 0.02	3.08 ± 0.10	–	13.10	35.60	0.020	0.96
batch	300	26.9 ± 0.3	30.68	0.83	1.21	1.09 ± 0.03	1.03 ± 0.06	3.58 ± 0.07	–	35.46	103.41	0.020	0.98
batch	1000	8.84 ± 0.22‡	104.04	0.92	1.09	1.00 ± 0.04‡	1.00 ± 0.03	3.92 ± 0.08	–	220.55	224.83	0.021	n/a
batch	3000	3.07 ± 0.08‡	317.30	0.97	1.03	0.98 ± 0.05‡	0.99 ± 0.03‡	4.01 ± 0.18	–	986.63	318.14	0.023	n/a

$t_{p,4}$ and w_4 in μs. n/a: with batching and long handlers the interpreter's switch interval interrupts a batch, so the recorded hold includes time waiting to resume. The rows without S_2 were measured in the supplementary run, at one and four workers. ‡ Runs behind this cell reported socket errors (at most 0.37% of a run's requests). † A configuration in this ratio had a coefficient of variation above 5%.

at four threads by less than a tenth (factors of 0.99 ± 0.06, 1.07 ± 0.03 and 1.02 ± 0.03). Batching removes the cost of the hand-offs, and what remains is the capacity of the lock as measured at four workers.

That capacity is smaller than the one-worker hold time suggests, and this is what closes the window. With 30 iterations and batching the lock is in use for 0.93 of every second, and still the threads scale by less than the bound of 2.59. The bound is computed from the hold time on one worker, 4.04 μs. A lock held that long per request could pass 247.8 thousand requests per second, which is what the four processes deliver (247.8). But with four threads the hold time is 5.91 μs, which allows 169.2 thousand, and the threads deliver 156.7. A growth of about one to two microseconds per call appears at every handler size up to 100 iterations (columns t_p and $t_{p,4}$ of Table 9); with longer handlers it is larger. It is not peculiar to threads. With one process per worker, where no interpreter state is shared, the hold time grows at least as much, from 3.95 to 7.20 μs with 30 iterations and from 1.22 to 2.71 μs with none. We therefore attribute it to the machine and not to the lock: at four workers no core is idle, every core also does kernel network processing, and our timer reads the wall clock, so whatever interrupts or slows a handler counts as hold time. We did not separate these causes. The narrowness of the window is thus partly a property of this testbed, in which workers and network processing share four cores, and not of $1/f$ alone; with cores to spare the hold time at four workers, and the window with it, may differ. Batching has a price of its own as well: no response from a batch is queued until all of its handlers have run, and with long handlers the interpreter's switch interval interrupts a batch part-way through. Processes behave in the opposite way and scale better as the handler grows, up to 4.00× and 4.01× for the two reactors. A larger share of each request is then processor time inside the worker, and a smaller share is kernel network processing.

Fig. 5b shows what this means for throughput, relative to the processes of the reactor with GIL batching. The process-per-worker baselines in it were measured in the same experiments. At four workers, batched `uringpy` threads deliver the following fractions of what the same reactor's processes deliver, as the handler's hold time on one worker grows from 1.21 through 1.71, 2.14 and 3.11 to 4.04 μs: 0.93, 0.88, 0.82, 0.70 and 0.63. Against the strongest baseline, `uvloop` processes, the same threads deliver 1.27, 1.20, 1.12, 0.94 and 0.85 times as much. With 100 added iterations the two ratios are 0.42 and 0.53, and with 300 they are 0.29 and 0.34. The window in which worker threads under the GIL can compete with a process per worker is therefore narrow, and it closes gradually: the ratio to processes falls at every step, as we had expected before the supplementary run. The threads are behind the runtime's own processes even with the shortest handler, by more than a tenth once the handler holds the GIL for about 1.71 μs and by more than a third at 4.04 μs. They stay ahead of the strongest baseline's processes up to about 2.14 μs and fall behind from about 3.11 μs. From 100 iterations on ($t_p \approx 10.93$ μs), every

Table 10: Requests served per GIL acquisition (`uringpy` with the handler, worker threads, GIL build). Cap: the largest number of requests handled under one acquisition; "—" is the reactor that takes the GIL for each request without the batching code, "none" takes it once for all requests found in a pass. Measured at four workers: GIL acquisitions per request $\hat{g}$, hold time $t_{p,4}$ and wait time w_4 per request in μs, GIL utilisation $U_4 = \bar{x}_4\, t_{p,4}$, voluntary context switches per request (c.sw.). Throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$).

Cap	$\hat{g}$	$\bar{x}_1$	$\bar{x}_4$	S_2	S_4	$t_{p,4}$	w_4	U_4	c.sw.
—	1.000	123.2 ± 2.0	193.9 ± 2.5	1.54 ± 0.04	1.57 ± 0.03	2.12	7.70	0.41	0.407
1	1.000	122.8 ± 2.5	190.2 ± 6.6	1.56 ± 0.04	1.55 ± 0.06	2.16	8.01	0.41	0.431
2	0.505	124.5 ± 2.6	221.5 ± 8.3	1.56 ± 0.05	1.78 ± 0.08	2.17	5.36	0.48	0.248
4	0.258	127.5 ± 2.1	245.8 ± 5.6	1.55 ± 0.04	1.93 ± 0.05	2.25	3.54	0.55	0.128
8	0.134	128.2 ± 2.2	262.2 ± 2.2	1.55 ± 0.04	2.04 ± 0.04	2.33	2.51	0.61	0.066
16	0.072	127.7 ± 3.5	271.0 ± 2.1	1.58 ± 0.05	2.12 ± 0.06	2.35	2.03	0.64	0.037
64	0.023	127.0 ± 2.3	279.6 ± 1.4	1.60 ± 0.04	2.20 ± 0.04	2.34	1.63	0.66	0.015
none	0.021	129.0 ± 1.5	280.3 ± 2.2	1.61 ± 0.05	2.17 ± 0.03	2.36	1.55	0.66	0.013

process-per-worker server we measured is faster than the threads. Several of the `asyncio` process configurations in Fig. 5b had a coefficient of variation above 5%; none of the ratios above uses them.

Free-threaded build with the handler. Without the GIL there are no hand-offs to save. `uringpy-app` threads scale by 2.37 ± 0.04 and deliver 1.00 ± 0.02 of what its processes do, and GIL batching makes no difference (1.00 ± 0.02). `asyncio-proto-app` threads scale by 2.51 ± 0.04, and the ratio between the two engines at four workers is 1.47 ± 0.01. Batched threads on the GIL build reach 0.96 of the throughput of threads on the free-threaded build.

The free-threaded series in Fig. 5a comes with a warning for longer handlers, though. Thread scaling there holds with 30 added iterations (2.36) and then falls, to 1.72 with 100, 1.26 with 300 and 1.03 with 3000. Under the GIL the last two are 1.05 and 0.97, so in scaling the free-threaded build is ahead by a fifth with 300 iterations and hardly at all with 3000. We ran a control without the runtime and saw the same pattern, more strongly. Plain Python threads calling the handler with 300 iterations complete 29.4, 39.3 and 21.7 thousand calls per second with one, two and four threads on the free-threaded build, and 33.7, 32.7 and 32.6 on the GIL build. The effect therefore appears without the runtime, whenever this handler, an interpreted integer loop, runs on plain threads of this interpreter version. We did not track down its cause, and it may lie in our handler as much as in the interpreter. Contention on the reference counts of objects that all threads use is a known source of such behaviour on this build [11], and we did not check whether the handler shares such objects. The result should not be taken as a statement about the free-threaded build. It does mean that the free-threaded results in Table 8 apply to the short handler measured there and may not carry over to longer ones.

Load. Everything so far was measured with 400 connections. Batch sizes, and with them the number of hand-off opportunities per request, depend on how many requests are waiting when a worker makes a pass, so in Table 11 we also measure with 1600, 64 and 16 connections.

With 1600 connections the picture is that of 400 connections. Batches are larger, so `uringpy-app-batch` takes the GIL 0.006 times per request and `py-uring` releases it 0.012 times, and the two scale by 2.29 ± 0.08 and 2.35 ± 0.08, like the C loop (2.33 ± 0.06). `py-epoll` stays at 0.64 ± 0.01 and the per-request reactor at 1.55 ± 0.07. By throughput at four threads the engines are in the order they have at 400 connections, which is what we had written down before the run; by S_4 the top two change places within their intervals. The one-worker figures at this load deserve less trust than the others. Every one-worker run of the four engines on `io_uring` reported socket errors, for less than 0.01% of its requests, probably because one worker accepting connections one at a time is slow to take 1600 of them from a listen queue of 1024, and their throughput is slightly below that at 400 connections, which raises S_4. Two things happen as the load gets lighter.

First, batches shrink and the GIL is given up more often. With 64 connections `uringpy-app-batch` takes the GIL 0.147 times per request, compared with 0.021 at 400 connections, and `py-uring` releases it 0.306 times per request, compared with 0.042. Their scaling drops to 1.64 ± 0.06 and 1.62 ± 0.05, while the C loop, which never takes the GIL, still reaches 1.90 ± 0.20. Within their confidence intervals the engines fall in the same order as at 400 connections. `py-epoll` loses throughput at every load (0.59, 0.62 and 0.64), and the reactor that takes the GIL per request gains almost nothing at 64 connections (1.08 ± 0.05). At four threads the batched reactor is still 1.44 ± 0.03 times faster than the per-request one with 64 connections, as it is with 400 (1.42 ± 0.11), but what it reaches is less. With 16 connections a pass finds so little to do that the advantage is nearly gone (1.06 ± 0.03), and `py-uring` releases the GIL 1.518 times per request, more than half as often as `py-epoll` does (2.538).

Second, with 16 connections the client no longer saturates the server. Even the C loop gains only 1.10 ± 0.14 from four workers and makes 0.422 voluntary context switches per request, although it never touches the GIL: its workers run out of work and sleep in the kernel. At

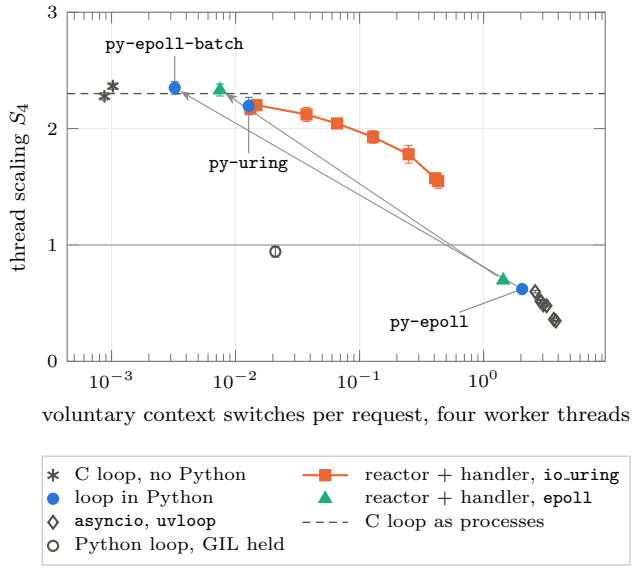


Figure 4: Thread scaling at four workers against voluntary context switches of the server per request (GIL build, 400 connections; bars are 95% half-widths; the grey line marks no gain over one worker, the dashed line the scaling of the C loop run as processes). Shown are the thread-mode configurations of Tables 4, 7, 8 and 10, an engine that appears in two of them once. The connected points are one reactor with the number of requests served per GIL acquisition varied. The arrows mark two interventions: the Python loop on `epoll` with its releases batched, and the `epoll` reactor taking the GIL per batch instead of per request. For `asyncio` and `uvloop` the request count is the client's.

Table 11: Load varied: keep-alive connections opened by the client (worker threads, GIL build). Mean throughput in thousands of requests per second at one and four workers ($n = 5$), scaling with its 95% half-width, and at four workers per request: counted GIL releases or acquisitions $\hat{g}$ and voluntary context switches (c.sw.).

Engine	Conn.	$\bar{x}_1$	$\bar{x}_4$	S_4	$\hat{g}$	c.sw.
`uringpy`	16	102.4	112.8	1.10 ± 0.14†	0	0.42
	64	141.9	270.1	1.90 ± 0.20†	0	0.03
	400	145.6	328.9	2.26 ± 0.08	0	0.00
	1600	141.7‡	330.5	2.33 ± 0.06	0	0.00
`py-uring`	16	92.6	88.4	0.95 ± 0.08†	1.52	0.98
	64	127.7	207.3	1.62 ± 0.05	0.31	0.17
	400	132.1	295.4	2.24 ± 0.04	0.04	0.01
	1600	127.3‡	299.0	2.35 ± 0.08	0.01	0.00
`py-epoll`	16	96.3	56.6	0.59 ± 0.04	2.54	2.28
	64	104.3	64.4	0.62 ± 0.03	2.08	2.17
	400	105.8	67.9	0.64 ± 0.03	2.01	2.01
	1600	103.6	66.0	0.64 ± 0.01	2.01	2.05
`uringpy-app`	16	87.2	88.0	1.01 ± 0.09†	1.00	0.70
	64	124.4	134.9	1.08 ± 0.05	1.00	0.67
	400	124.3	198.5	1.60 ± 0.13†	1.00	0.39
	1600	119.1‡	184.8	1.55 ± 0.07	1.00	0.43
`uringpy-app-batch`	16	90.3	93.2	1.03 ± 0.04	0.60	0.69
	64	118.5	194.0	1.64 ± 0.06	0.15	0.16
	400	128.7	281.4	2.19 ± 0.02	0.02	0.01
	1600	122.0‡	279.5	2.29 ± 0.08	0.01	0.01

The rows with 1600 connections were measured in a later run with the server's limit on open files raised; 50 earlier runs, made under a limit below that number of connections, are not shown (Section 7). ‡ Every run of this configuration reported socket errors, for at most 0.008% of its requests. † A configuration in this ratio had a coefficient of variation above 5%.

16 connections, what limits the engines that scale elsewhere is therefore mainly the offered load. These rows also show the limit of context switches as an indicator, which is why we use them only under saturating load (Section 5.3). They are also the nearest we come to a server that is not saturated, and Table 11 has no process-mode reference for them, so we cannot say how threads compare with processes below saturation.

Response size. Table 12 varies the size of the response body, with four worker processes on the GIL build against `asyncio`'s Protocol API. With bodies of 64 bytes and of 1 KiB `uringpy` is 1.41× and 1.41× faster. From 16 KiB upward the two cannot be told apart: the client receives about 15.7 Gbit/s whichever engine is serving, so the network path of the testbed is the bottleneck and those rows no longer measure the servers. We did not investigate what limits that path, and we measured no size between 1 and 16 KiB.

6 Discussion

Attributing a speed-up. If a Python loop on `io_uring` is faster than `asyncio`, that does not by itself show that `io_uring` is the reason. On one worker the interface is worth a factor of 1.15–1.25 in the two pairs of the design. The saving from fewer system calls is almost used up at 16 completions per call, and a bare C loop is only 1.08–1.17× faster than the same loop in Python. A figure of several hundred completions per system call is real, but it says little about single-worker throughput. One `asyncio` worker using the streams API is 2.14× slower than one `uringpy` worker. Of that, a factor of 1.59 lies between `asyncio` and our bare Python loop on the same interface, and so belongs to what a full event loop does for each event. The interface and the language of the dispatch share the rest. Against `asyncio`'s Protocol API the corresponding factor is 1.15, no larger than either of the other two.

What batching buys. For worker threads under the GIL, the valuable thing about batching is that the lock is given up once per batch, where it would otherwise be given up at every system call or every request. A completion interface makes this the natural way to write a loop, which is one reason a loop on `io_uring` can scale across threads where the usual readiness loop collapses. But the benefit belongs to the batching. An `epoll` loop and an `epoll` reactor that batch their releases scale as well as their `io_uring` counterparts, and an `io_uring` loop that holds the GIL across its submitting call does not scale

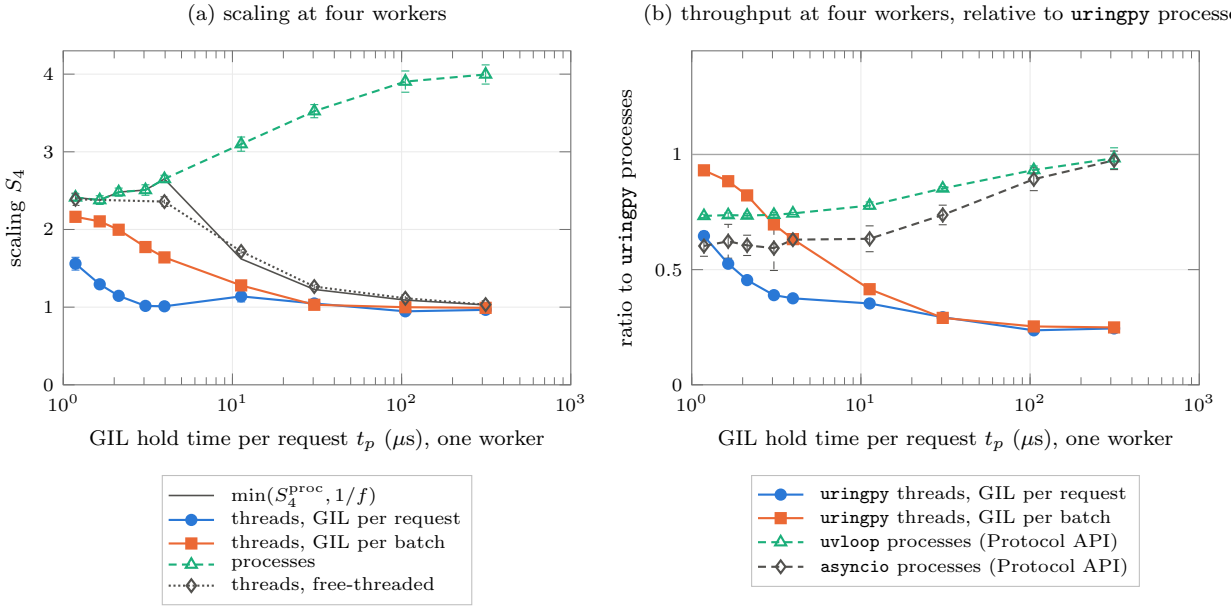


Figure 5: Handler cost swept (GIL build unless stated). The horizontal axis is the measured time one request holds the GIL on one worker. (a) Scaling at four workers. The line is the smaller of the bound $1/f$ of (1) and the scaling the same workers reach as processes. The free-threaded series has no points for the three handler sizes added in the supplementary run. (b) Throughput at four workers relative to `uringpy` processes with GIL batching (grey line): worker threads of one interpreter (solid) and the two baselines run as one process per worker through the Protocol API (dashed). Bars are 95% half-widths.

Table 12: Response-body size (four worker processes, GIL build). Throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$); for `uringpy` also the data rate received by the client, busy server cores and system calls per request.

Body	`uringpy`	`asyncio-proto`	Gbit/s	C	sys/req
64 B	325.9 ± 1.0	231.8 ± 1.2	0.4	3.91	0.02
1 KiB	319.5 ± 0.6	227.4 ± 1.6	2.9	3.91	0.02
16 KiB	119.2 ± 0.6	118.8 ± 1.0	15.7	3.29	0.65
64 KiB	30.0 ± 0.2	30.0 ± 0.1	15.8	1.43	0.72
256 KiB	7.51 ± 0.01	7.50 ± 0.03	15.8	1.26	1.16
1 MiB	1.86 ± 0.01	1.87 ± <0.01‡	15.7	1.62	2.45

‡ Runs of this configuration reported socket errors (less than 0.01% of a run's requests).

at all. The interface is not irrelevant either. When the GIL is taken for every request, the `io_uring` reactor loses much less than the `epoll` reactor, apparently because it gives waiting threads less opportunity to take the lock. Counting releases therefore does not settle how a design will behave. Among the configurations of Fig. 4, voluntary context switches per request were the better guide in our data, and any server can report them. They are a guide under saturating load only: with 16 connections the C loop makes 0.422 per request without touching the GIL. An `asyncio`-compatible loop on `io_uring` could fall anywhere in that figure, depending on where it releases the GIL and how much Python it runs per event.

Three ways to keep hand-offs rare. The runtime offers a reactor that runs no Python, a reactor that calls a handler under one GIL acquisition per batch, and a batch interface for loops written in Python. On the transport path the first is 1.12× faster than the third at four workers, so its advantage over a well-behaved Python loop is modest. With a short handler, the second makes a larger difference. Per-request locking delivers 0.92 of the best process-per-worker baseline, and per-batch locking delivers 1.28 of it, still inside a single interpreter.

The limits. Hand-offs are an overhead that a design can avoid, given a backlog of requests to batch. The time the handler itself holds the GIL is not. With a handler of about a microsecond, batched threads reach 0.93 of what their own processes deliver. The fraction falls steadily as the handler grows, to less than two thirds at about four microseconds, and the gap widens from there for every design we measured (Fig. 5b). The same numbers say that this result belongs to four cores. With the short handler, four batched threads keep the lock busy for 0.66 of the time. At the hold time measured there the lock can pass at most 419.0 thousand requests per second, and the four threads already deliver 278.2. Processes have no such ceiling, so on a machine with more cores we would expect the threads to stop near it while processes went on. We could not test this. A handler that parses a request and dispatches into a framework will usually hold the GIL for longer than that, although we did not measure one. For such handlers our result is negative: under the GIL one needs processes, sub-interpreters, or handlers that release the lock themselves. With fewer connections there

is less to batch, and thread scaling falls; with more, it holds (Table 11). On the free-threaded build the hand-off problem disappears and the runtime keeps only an advantage in speed per worker. Handlers of 100 iterations and more scaled poorly there as well, for reasons that we did not identify and that may lie in our handler.

Threads or processes. Worker processes reach at least the throughput of `uringpy`'s worker threads, so speed is not an argument for threads. The argument is that the workers live in one interpreter and can share state without inter-process communication. We did not measure a workload that depends on shared state. What this paper shows, on one machine under saturating load, is that the option cost nothing in throughput on the transport path and, with a handler of about a microsecond, left the threads at 0.93 of the processes' throughput. How much a particular application would gain from it remains open, and so does the behaviour of a server that is not saturated, which is the normal operating point: batching needs a backlog, and below saturation there is little of one.

7 Threats to Validity

Table 13 lists the ways in which a measurement of this kind can mislead, and what we did about each. The limitations that remain are discussed below.

Testbed. The server has four cores, so we cannot show scaling beyond four workers. At four workers every core is busy and the kernel's network processing competes with the workers (E_4 in Table 7 quantifies this), so that on the transport path and with the short handler even processes scale by only 2.26 to 2.72 (both builds, leaving out configurations with a coefficient of variation above 5%). Differences between configurations that all reach this ceiling are compressed, and a statement such as "threads match processes" means that both reach the ceiling of this machine. At two workers, where cores are still idle, the separation is already there for the loops (Section 5.5) and for the `epoll` reactor (Section 5.7), but for the `io_uring` reactor the gain from GIL batching is small. We do not know whether the picture of Fig. 4 holds with more cores, where more threads compete for the GIL, although the model gives no reason to expect hand-offs to get cheaper. The machines are cloud virtual machines of one family, running one kernel and one CPython version (3.14.7), with workers not pinned to cores and stock interrupt settings, so quantities such as τ and the growth of the hold time are specific to this testbed. All runs were made on one pair of machines within a single day, so the confidence intervals describe the variation within that session only. The free-threaded results describe that version of an implementation that is still changing. With responses of 16 KiB and more the network path was the bottleneck.

Workload. The workloads consist of one protocol exchange with a fixed or lightly computed response. Requests are not parsed, and we do not cover TLS, request bodies or realistic application logic. The transport workload runs no Python in `uringpy` and runs Python for every event in the baselines, which is the most favourable case for the runtime. Load comes from a closed-loop generator over keep-alive connections, so every throughput in this paper is that of a saturated server, and the factors by which threads lose throughput are specific to saturation. Table 11 varies the number of connections between 16 and 1600 but not the arrival pattern, and we did not measure open-loop or bursty load. With 16 connections the client did not saturate the server. We use `io_uring` in its basic form (Section 4) and did not evaluate features such as multishot receive or deferred task running. The Python loops of the 2×2 design are bare, so the loop-placement effect on one worker is a lower bound for a full event loop. The handler is one small function, optionally lengthened by an interpreted loop. Handlers that release the GIL themselves, for I/O or inside native code, were not measured and would behave differently.

The model. The model was formulated after the first data, and the data support (2) as an ordering and not as a predictor. Hand-offs are not observed. We count releases and acquisitions ($\hat{g}$), an upper limit, for our own engines only. The cost τ is a residual of throughput and hold time under the assumption that the bound holds with equality, so it cannot test the model, and no single value fits every configuration: it ranges from 2.9 to 5.4 μs over the handler sizes for which it is precise to a microsecond. The bound (1) is above this machine's ceiling for short handlers. With 30 iterations, where it is about what processes reach, it is 58% above the scaling of the batched reactor, and with 100 iterations and more it is still 4–25% above. For the Python loops f is estimated from one-worker throughputs and not measured. The release control lowers f as well as $\hat{g}$, and the hold-time control halves $\hat{g}$ while raising f (Section 5.5). The reactor pair changes the order of work within a pass as well as the number of acquisitions. Only the cap sweep changes a single quantity, and it spans a smaller range of scaling than the collapse it is meant to explain. Voluntary context switches indicate how many hand-offs put a thread to sleep, but they are an outcome of contention and not an independent variable, they are divided by the requests served, and they include waits for I/O. We rely on them only under saturating load, where workers with their own GIL make almost none, to group configurations and to follow a trend within one engine. For `asyncio` and `uvloop` they are divided by an estimated request count. Our explanation of why the two per-request reactors differ rests on these counters and on the structure of the loops. The batch cap for system calls was varied only for the C reactor on one worker.

What was not measured. We could not measure the `io_uring` event loops that motivated the study, although we tried on the same machines, in an image derived from the one the baselines ran in. Both failed on CPython 3.14, a version newer than those `uringcore` states it is tested

on (3.10 to 3.13). `uringloop` (0.1.0), which asks for Python 3.12 or newer, did not start, because it depends on a part of `asyncio` that Python 3.14 has removed. `uringcore` (0.9.1) started once the container's limit on locked memory was raised and answered requests, but stopped answering after a number of requests close to the size of its buffer pool, with the default pool and with a larger one, and reported that no buffers were available. All 30 runs of it in the supplementary run ended with the client unable to connect, which we attribute to the same behaviour. Those 90 runs, 60 of them repetitions of `asyncio` and `uvloop` under the same script, are kept with the superseded runs, together with a record of these checks. An earlier attempt under the default limits started neither loop; its 60 runs of `asyncio` and `uvloop` are not used either. We did not investigate further and make no claim about either loop on the versions they support. We reproduced the slowdown of the interpreted loop across threads on the free-threaded build without the runtime, but did not explain it. We measured no production server, no sub-interpreters, and no workload that benefits from state shared between workers.

Measurement. Latency comes from a closed-loop generator and is not an estimate of tail latency; the latency columns are means of per-run 99th percentiles. The contention probe is usable only with one competing thread on this machine (Section 3). 14 of the 428 configurations had a coefficient of variation above 5%, and those that enter a ratio in a table are marked there. Socket errors were reported in three places. The first is the handler sweep with 1000 or 3000 added iterations (107 runs): for our reactors on one and two workers and, with 3000 iterations, on four, and for the `uvloop` baseline as one process. The second is the one-worker runs of the four `io_uring` engines with 1600 connections (20 runs), whose throughput is the denominator of S_4 in those rows of Table 11. The third is 6 other runs: the `uvloop` baseline as one process with 300 iterations, and one run with 1 MiB responses. The driver records only the total that `wrk` prints and not the kind of error, so we cannot say what they were. They affect at most 0.37% of a run's requests. One test in the runtime's test suite failed once on the free-threaded image in an earlier revision and has passed in every run since. We did not find the cause, and it may point to a race on that build. One baseline also deserves a warning. On one worker, `uvloop` with the streams API has a 99th-percentile latency of 110.0 ms, against 6.1 ms for `asyncio` at a similar throughput. We did not find out why. With the Protocol API, which the comparisons we rely on use, the two loops are alike (8.3 and 8.4 ms).

Excluded and repeated runs. The main run included the connection sweep at 1600 connections, and we excluded those 50 runs. The container limited a process to 1024 open files, fewer than the connections offered. The `py-epoll` server stopped on the first `accept` that failed, in all 10 of its runs. The other servers kept running but served only the connections they had been able to accept: throughput times mean latency puts these at between 947 and 1016 in every surviving run. None of these runs therefore measured 1600 connections. They remain in the repository, and the exclusion is a rule in the script that generates the tables, which counts them. Apart from this exclusion and the 90 runs of the failed `uringcore` attempt, every run of the main and the supplementary run is in the data set. The 1600-connection rows of Table 11 come from the supplementary run, in which the limit was raised. Those rows and the three added handler sizes were measured about half a day after the configurations they are compared with, in separate experiments, and no configuration of these two experiments was measured in both runs. Each ratio we report for them is formed within one experiment, but a comparison of absolute throughput between a supplementary row and a main-run row carries the uncertainty described next.

Comparisons across experiments. Repetitions are interleaved within an experiment but not between experiments. One `uringpy` worker thread on the GIL build with 400 connections was measured in four of them, at 144.8 ± 2.3, 144.3 ± 1.9, 143.0 ± 3.2 and 145.6 ± 5.0 thousand requests per second. Ratios between configurations of the same experiment are not affected by this. Comparisons that span two experiments carry the additional uncertainty, and where we give intervals for them they reflect only the variation within each experiment. This applies to the capped `io_uring` loop against `epoll`, to `py-epoll` against `asyncio`, to the cost of the handler on one worker, to timed against untimed runs, and to every comparison between the GIL build and the free-threaded build.

Hypothesis and analysis. We formulated the central statement after seeing the first data (Section 5.4). Of the experiments that test it, one has its prediction on record from before the run, and that control is the least clean of the three. The expectations recorded for the supplementary run were weak ones. Apart from that, the analysis did not follow a pre-registered plan. The earlier measurements, including those that led us to a wrong conclusion, are kept in the repository.

8 Conclusion

What batching buys a Python server depends on how many threads share the interpreter. For a single worker it saves system calls. Together with the interface this is worth a factor of 1.15–1.25, and most of it has been collected by the time a batch holds 16 completions. For several worker threads under the GIL, batching does something more useful: the lock is handed over once per batch, where it would otherwise change hands at every system call or every request. In our measurements this is what separates designs that scale from designs that lose throughput, and the interface and the language of the loop matter mainly through it. An `epoll` reactor with a

Table 13: Pitfalls, what was done about each, and what was not.

Pitfall	What is done	Evidence
The load generator, not the server, is the limit	The client has twice the server's cores; client processor use is recorded for every run and configurations above 85% are flagged	Section 5.5; raw data
Noise mistaken for an effect	Five repetitions, interleaved in shuffled order, a fresh server per run, 95% confidence intervals; cross-experiment spread stated. One session on one pair of machines	every results table
A weak baseline	`asyncio` through both its streams and its faster Protocol API, and `uvloop`; with the handler, `asyncio` through both APIs and `uvloop` through the Protocol API; process mode as well as thread mode, and process mode in the handler sweep; one interpreter version, one listen backlog and `TCP_NODELAY` for every engine. No production server	Tables 7, 8; Fig. 5
GIL and free-threaded builds differing in more than the GIL	Both interpreters built from one recipe; the GIL state reported by the server is recorded per run	Table 2
Crediting `io_uring` for what C code does, or the reverse	The 2×2 design varies the two separately; the batch cap varies batching alone	Tables 4–6
Crediting the interface or the loop's language for what the handling of the GIL does	Three interventions, each aimed at one quantity: GIL releases in a Python loop, time under the GIL in a Python loop, and GIL acquisitions in a reactor with the handler fixed. Only the cap sweep changes nothing else. The loops are also run without the GIL	Tables 4, 8, 10
A correlation across different engines read as a cause	One engine with the number of requests per GIL acquisition varied in steps; the figure across engines is presented as description only	Table 10; Fig. 4
Runs dropped without saying so	The runs outside the data set are the 1600-connection runs of the main run and those of the attempts to measure `uringcore`; they are counted and explained	Section 7
A one-worker baseline that also uses other cores	Server processor use recorded; busy cores and throughput per busy core reported next to scaling	Table 7
A model fed with assumed numbers	GIL hold time and wait time measured per request at nine handler sizes. Hand-offs are bounded by a count and not measured	Table 9
Results that hold at one load only	Connections varied from 16 to 1600. Closed-loop load only	Table 11
Engines doing different protocol work	Every engine answers each `recv` with identical bytes; none parses HTTP	Section 5
Errors hidden in a throughput figure	Socket errors, non-2xx responses, handler errors and failed runs recorded and reported	Sections 5.5, 7
Code changing between experiments	All experiments run on images built at one commit, after the test suite has passed in both; the supplementary run reuses those images and changes only two container limits	Section 5
Numbers not traceable to code, or mistyped	Commit, image, interpreter and seed recorded per experiment; raw runs archived, superseded runs included; tables, figures and the benchmark numbers in the text generated from them	repository

Python handler scales by 2.33× on four cores if it takes the GIL once per batch and falls to 0.70× if it takes it for every request, with the I/O system calls and the handler unchanged. A Python loop on `epoll` behaves the same way (2.35× against 0.62×), and a Python loop on `io_uring` falls from 2.19× to 0.94× if it merely holds the GIL during its submitting call. Without the GIL all of them scale alike.

A runtime built on this observation lets one interpreter on a standard build match a process per worker when no Python runs per request. With a Python handler of about a microsecond, taking the GIL once per batch brings its threads to 0.93 of the throughput of its processes. The window is narrow. That fraction falls steadily with the length of the handler, to 0.63 at about four microseconds, and none of the designs we measured restores thread scaling under the GIL beyond that. With fewer connections there is less to batch, and thread scaling falls. On the free-threaded build `asyncio` threads scale as well, at least with a short handler, and what is left is a difference in speed per worker.

On this testbed and under saturation, then, the cost that batching most needs to spread out for worker threads of one CPython interpreter is the GIL hand-off, and the system call comes second. The statement rests on one four-core server, on closed-loop load, on synthetic handlers, and on hand-offs that were inferred from counters and interventions and not observed.

Several questions remain open. We measured on four cores, and it would be worth knowing how the cost of a hand-off behaves when more threads compete for the lock, and to trace the lock itself. We could not benchmark the existing `io_uring` event loops on CPython 3.14, and a study of where each of them releases the GIL would show which regime it is in. Finally, our handlers were synthetic. A server that parses HTTP in C and enters Python only for application code is the natural next step, and it would show how much of a realistic request can be kept outside the lock.

Acknowledgment

[[PENDING: Author to confirm the wording before submission. IEEE requires content generated by AI to be disclosed in this section.]] The author used Claude (Anthropic), an AI assistant, in revising this work: to review an earlier draft and propose additional experiments in response; to write parts of the implementation (the application-handler reactor, GIL batching, the `epoll` reactor and its batch helper, the control loops, the benchmark driver, the generator of tables, figures and quoted numbers, and the tests) and of the documentation; and to draft text in every section of this paper, including the abstract. The author ran all experiments, reviewed and tested this content, and is responsible for the paper's claims.

Code and Data Availability

The runtime and all benchmark scripts are open source (MIT) at `https://github.com/marimuthuvm/uringpy` and released on PyPI (`pip install uringpy`). The repository contains the raw output of every run reported here and of the earlier, superseded runs (`benchmarks/results/`), the script that generates the tables, the figures and the numbers quoted in the text from it (`benchmarks/make_tables.py`), the scripts that ran the main and the supplementary run (`benchmarks/run_paper_experiments.sh` and `benchmarks/run_supplement.sh`), the container recipe for both interpreters, and a description of each experiment (`benchmarks/EXPERIMENTS.md`). **[[PENDING: Archive the release that contains these results and cite its DOI here; the earlier archive, `https://doi.org/10.5281/zenodo.23129560`, predates them.]]**

Conflict of Interest

The author has no financial or personal relationships that could have influenced this work, and declares no conflict of interest. No external funding was received.

References

[1] J. Axboe, "Efficient IO with io_uring," 2019. [Online]. Available: `https://kernel.dk/io_uring.pdf` (accessed Oct. 7, 2026).

[2] "uringloop: A liburing-based proactor event loop for asyncio." [Online]. Available: `https://github.com/bright2227/uringloop` (accessed Oct. 7, 2026).

[3] A. K. Pandey, "uringcore: High-performance asyncio event loop for Python using io_uring." [Online]. Available: `https://github.com/ankitkpandey1/uringcore` (accessed Oct. 7, 2026).

[4] L. Soares and M. Stumm, "FlexSC: Flexible system call scheduling with exception-less system calls," in *Proc. 9th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2010, pp. 33–46.

[5] S. Han, S. Marshall, B.-G. Chun, and S. Ratnasamy, "MegaPipe: A new programming interface for scalable network I/O," in *Proc. 10th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2012, pp. 135–148.

[6] D. Didona, J. Pfefferle, N. Ioannou, B. Metzler, and A. Trivedi, "Understanding modern storage APIs: A systematic study of libaio, SPDK, and io_uring," in *Proc. 15th ACM Int. Systems and Storage Conf. (SYSTOR)*, 2022, pp. 120–127, doi: 10.1145/3534056.3534945.

[7] M. Jasny, M. El-Hindi, T. Ziegler, V. Leis, and C. Binnig, "High-performance DBMSs with io_uring: When and how to use it," *Proc. VLDB Endow.*, vol. 19, no. 9, pp. 2317–2330, 2026, doi: 10.14778/3819518.3819553.

[8] D. Beazley, "Understanding the Python GIL," presented at PyCon, Atlanta, GA, USA, 2010. [Online]. Available: `http://www.dabeaz.com/GIL/` (accessed Oct. 7, 2026).

[9] M. W. Blasgen, J. Gray, M. F. Mitoma, and T. G. Price, "The convoy phenomenon," *ACM SIGOPS Oper. Syst. Rev.*, vol. 13, no. 2, pp. 20–25, Apr. 1979.

[10] E. Snow, "PEP 684: A Per-Interpreter GIL," Python Enhancement Proposals, 2022. [Online]. Available: `https://peps.python.org/pep-0684/` (accessed Oct. 7, 2026).

[11] S. Gross, "PEP 703: Making the Global Interpreter Lock Optional in CPython," Python Enhancement Proposals, 2023. [Online]. Available: `https://peps.python.org/pep-0703/` (accessed Oct. 7, 2026).

[12] T. Wouters, M. Page, and S. Gross, "PEP 779: Criteria for supported status for free-threaded Python," Python Enhancement Proposals, 2025. [Online]. Available: `https://peps.python.org/pep-0779/` (accessed Oct. 7, 2026).

[13] K. Aditya, "Scaling asyncio on free-threaded Python," Quansight Labs, Sep. 2025. [Online]. Available: `https://labs.quansight.org/blog/scaling-asyncio-on-free-threaded-python` (accessed Oct. 7, 2026).

[14] S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D. S. Seljebotn, and K. Smith, "Cython: The best of both worlds," *Comput. Sci. Eng.*, vol. 13, no. 2, pp. 31–39, 2011.

[15] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proc. AFIPS Spring Joint Computer Conf.*, vol. 30, 1967, pp. 483–485.

[16] S. Eyerman and L. Eeckhout, “Modeling critical sections in Amdahl’s law and its implications for multicore design,” in *Proc. 37th Annu. Int. Symp. Computer Architecture (ISCA)*, 2010, pp. 362–370, doi: 10.1145/1815961.1816011.

[17] MagicStack, “uvloop: Ultra fast asyncio event loop.” [Online]. Available: `https://github.com/MagicStack/uvloop` (accessed Oct. 7, 2026).

[18] “Granian: A Rust HTTP server for Python applications.” [Online]. Available: `https://github.com/emmett-framework/granian` (accessed Oct. 7, 2026).

[19] A. Georges, D. Buytaert, and L. Eeckhout, “Statistically rigorous Java performance evaluation,” in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2007, pp. 57–76, doi: 10.1145/1297027.1297033.

[20] W. Glozer, “wrk: Modern HTTP benchmarking tool.” [Online]. Available: `https://github.com/wg/wrk` (accessed Oct. 7, 2026).

[21] B. Schroeder, A. Wierman, and M. Harchol-Balter, “Open versus closed: A cautionary tale,” in *Proc. 3rd USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2006.

Marimuthu Velayutham is an independent researcher working on high-performance systems software and language runtimes, with interests in asynchronous I/O, operating-system interfaces, and the performance of managed-language runtimes.